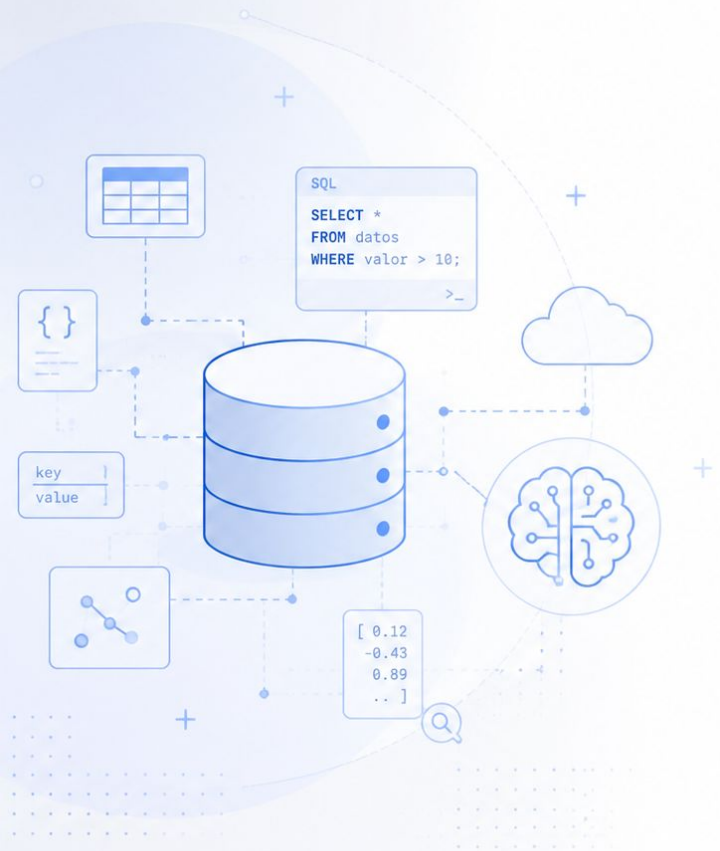




Bases de Datos para Inteligencia Artificial

Especialización en Inteligencia Artificial

Docente:

Lacheski, Martín Anibal

Clase 7:

Seguridad Aplicada, Tecnologías Especializadas y Buenas Prácticas

Bases de Datos para IA

No alcanza con almacenar correctamente los datos

En las clases anteriores trabajamos con:

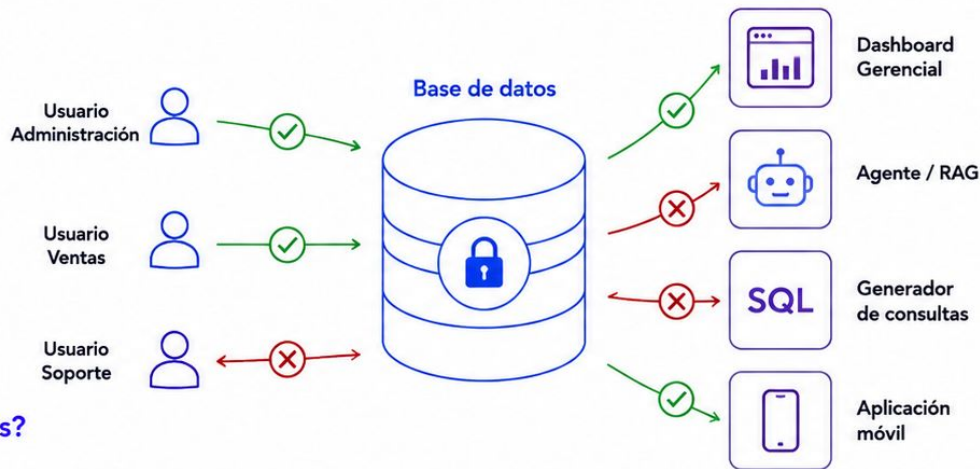
- modelado relacional;
- SQL y optimización;
- bases NoSQL;
- arquitecturas distribuidas;
- Data Warehouse, Lake y Lakehouse;
- bases vectoriales;
- transacciones y concurrencia;
- replicación y escalabilidad.

Ahora aparece otra pregunta:

¿quién puede acceder a cada dato y bajo qué condiciones?

En aplicaciones tradicionales esto ya era importante.

Cuando incorporamos agentes, RAG o generación automática de consultas, el problema se vuelve todavía más relevante.



Una consulta técnicamente correcta también puede ser incorrecta si devuelve datos que el usuario no debería ver.

Bases de Datos para IA

Agenda del día

- Seguridad aplicada a bases de datos.
- Autenticación, autorización, roles y privilegios.
- Principio de mínimo privilegio.
- Arquitecturas multi-tenant.
- Estrategias de aislamiento.
- Row Level Security en PostgreSQL.
- Contexto de usuario y tenant.
- Acceso desde aplicaciones.
- RAG y filtros de seguridad.
- Text-to-SQL y agentes.
- Riesgos de acceso indirecto a datos.
- Tecnologías especializadas.
- Selección tecnológica.
- Buenas prácticas.
- Práctica guiada con PostgreSQL y RLS.



Usuario



Seguridad



PostgreSQL



Rol



Tenant



RAG



Aislamiento



Row Level
Security (RLS)



SQL



Acceso desde
aplicaciones



Text-to-SQL
y agentes



Tecnologías
especializadas



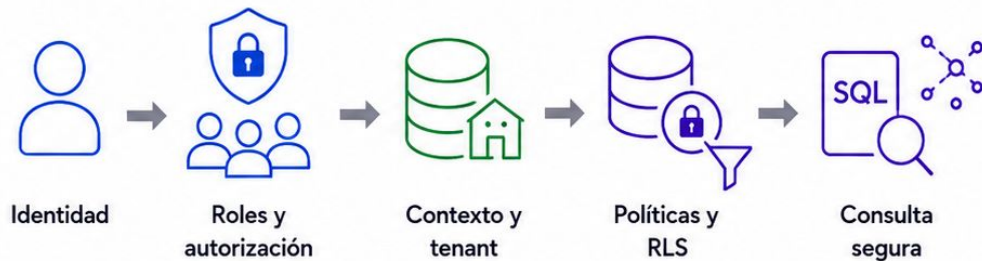
Hoy vamos a recorrer cómo proteger, aislar y habilitar accesos seguros en sistemas de datos conectados a aplicaciones y agentes.

Objetivos de la clase

objetivos

Al finalizar la clase, deberíamos poder:

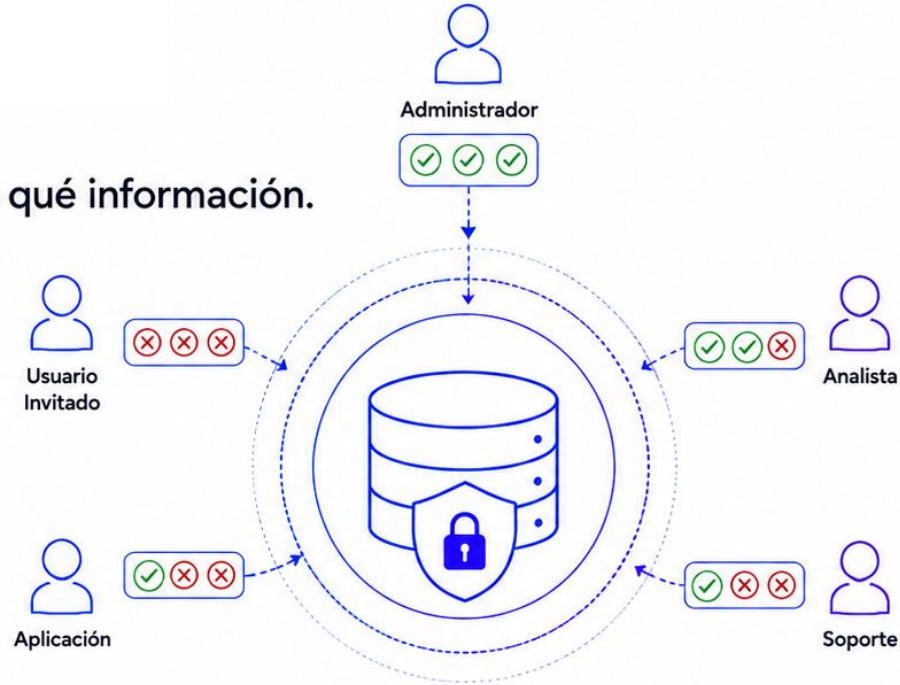
- diferenciar autenticación de autorización;
- trabajar con roles y privilegios;
- aplicar el principio de mínimo privilegio;
- reconocer alternativas para una arquitectura multi-tenant;
- implementar aislamiento mediante tenant_id;
- comprender Row Level Security;
- crear políticas de acceso en PostgreSQL;
- propagar de forma segura el contexto del usuario;
- proteger búsquedas vectoriales y sistemas RAG;
- reconocer riesgos de Text-to-SQL;
- seleccionar tecnologías según el tipo de dato;
- aplicar buenas prácticas de diseño y consulta.



La seguridad debe formar parte del modelo y de la arquitectura, no agregarse únicamente al final.

Seguridad como Problema

Definir quién puede acceder a qué información.



La seguridad empieza por definir qué datos puede ver cada usuario.

Seguridad de datos

No se trata solamente de proteger el servidor

Una solución de datos necesita controlar:

- quién puede conectarse;
- qué objetos puede consultar;
- qué filas puede observar;
- qué columnas puede modificar;
- qué operaciones puede ejecutar;
- desde qué aplicación;
- con qué identidad;
- durante cuánto tiempo;
- qué acciones quedan registradas.



Proteger una base implica controlar conexiones, operaciones y datos visibles.

Dos preguntas diferentes

Quién sos y qué podés hacer

1. Autenticación

Responde:

“¿quién está intentando acceder?”

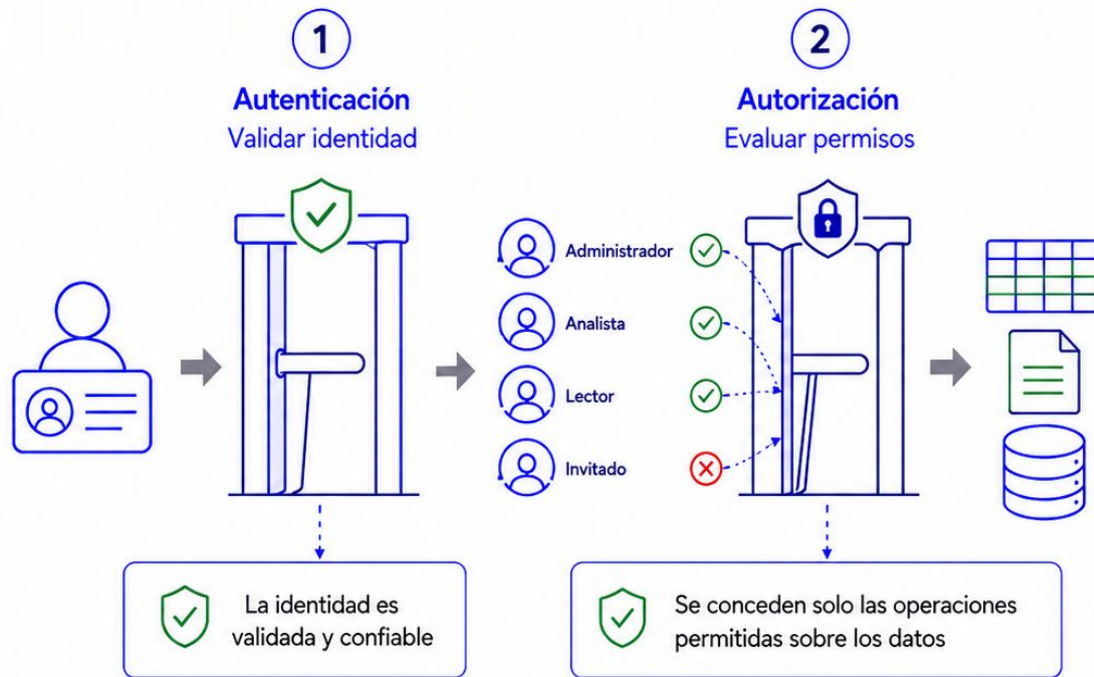
Puede utilizar:

- usuario y contraseña;
- certificados;
- tokens;
- proveedores de identidad;
- credenciales de servicio.

2. Autorización

Responde:

“¿qué operaciones puede realizar esa identidad?”



Estar autenticado no significa tener permiso para acceder a todos los datos.

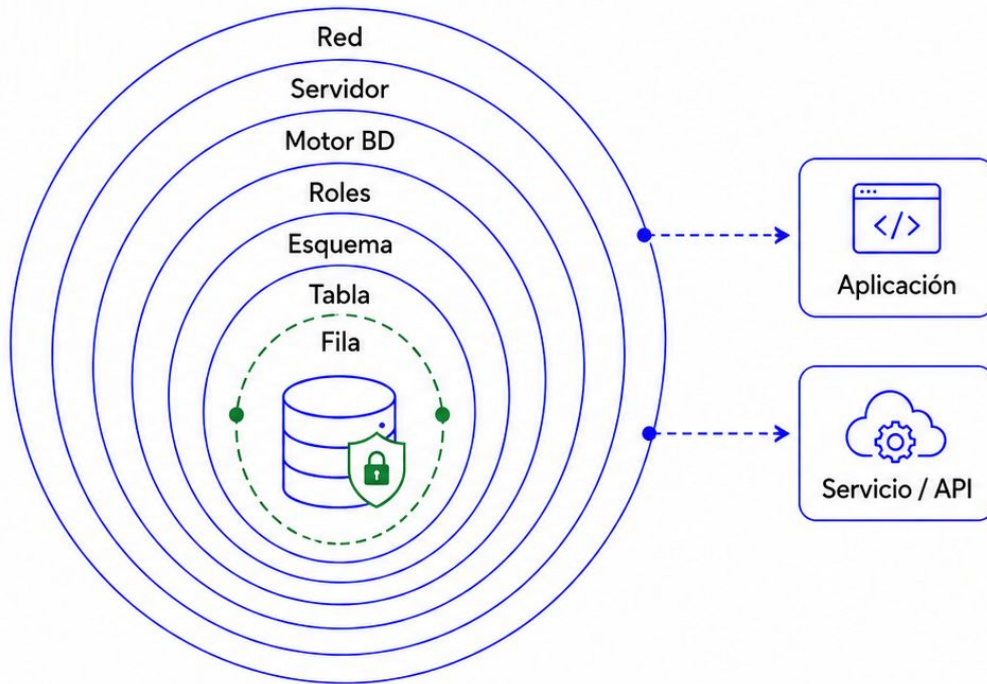
Defensa en profundidad

No depender de un único control

Podemos aplicar seguridad en diferentes niveles:

1. red;
2. servidor;
3. motor de base de datos;
4. roles y privilegios;
5. esquema;
6. tabla;
7. fila;
8. aplicación;
9. servicio o API.

Si una capa falla, otra todavía puede limitar el acceso.



Una arquitectura segura evita depender únicamente de que la aplicación recuerde aplicar un filtro.

Acceso indirecto

El usuario ya no siempre escribe la consulta

En una aplicación tradicional:

Usuario → aplicación → consulta SQL.

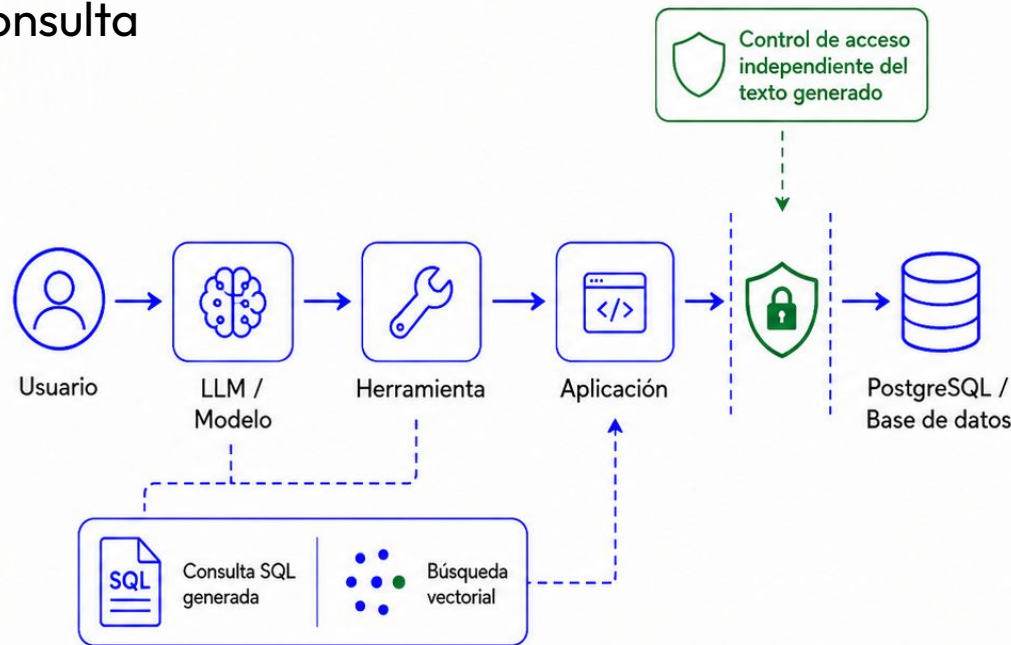
En una aplicación basada en IA puede aparecer:

Usuario → modelo → herramienta → aplicación → base.

La consulta puede ser:

- generada automáticamente;
- modificada por un agente;
- construida a partir de lenguaje natural;
- acompañada por búsquedas vectoriales.

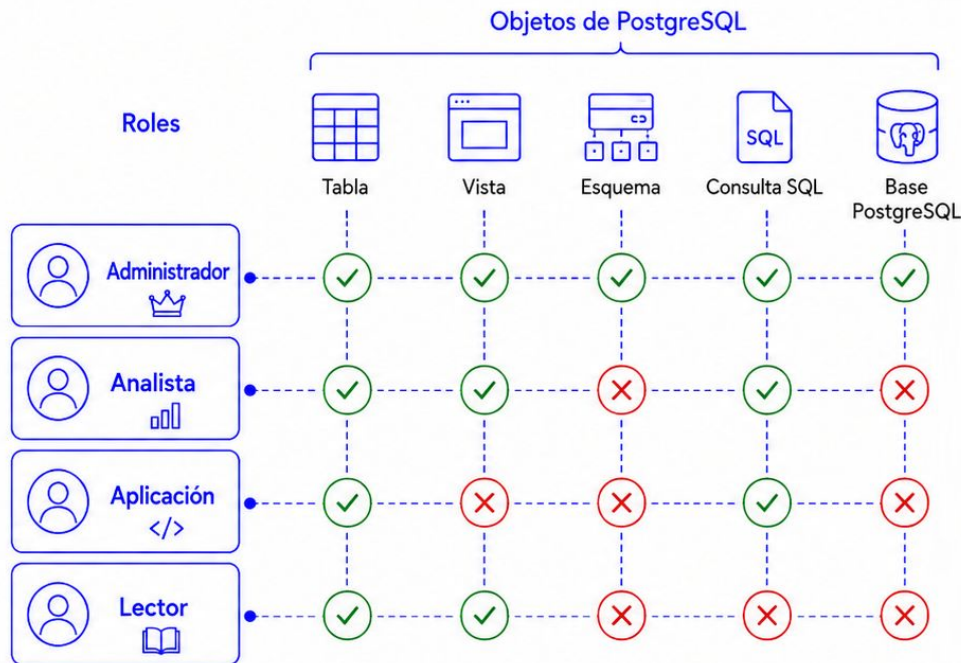
Esto aumenta la importancia de aplicar controles independientes del texto generado.



El modelo puede proponer una consulta. La base debe seguir aplicando las reglas de acceso.

Roles y Permisos

Controlar operaciones
antes de controlar filas.



Los roles permiten definir qué operaciones puede realizar cada identidad sobre los objetos de la base.

Roles

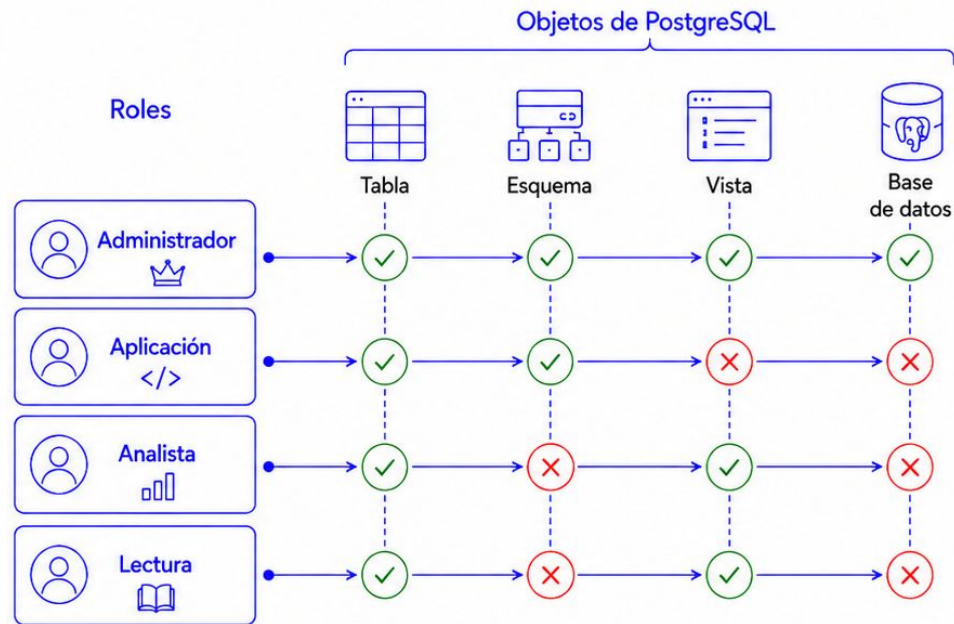
Identidades y conjuntos de permisos

PostgreSQL utiliza roles para representar:

- usuarios;
- aplicaciones;
- administradores;
- procesos automáticos;
- grupos de permisos.

Un rol puede:

- iniciar sesión;
- poseer objetos;
- recibir privilegios;
- heredar privilegios;
- pertenecer a otros roles.



Conviene asignar permisos según responsabilidades, no compartir una cuenta general para toda la organización.

GRANT y REVOKE

Autorizar operaciones explícitamente

GRANT SELECT
ON documentos
TO analista;

GRANT SELECT, INSERT, UPDATE
ON fragmentos
TO aplicacion;

REVOKE DELETE
ON fragmentos
FROM aplicacion;

Los permisos pueden limitar:

- lectura;
- inserción;
- modificación;
- eliminación;
- ejecución de funciones;
- uso de esquemas.



Matriz de permisos por rol y operación

	 SELECT	 INSERT	 UPDATE	 DELETE	 EXECUTE	 USAGE
 analista	✓	✗	✗	✗	✗	✗
 aplicacion	✓	✓	✓	✗	✗	✓
 lectura	✓	✗	✗	✗	✓	✓



No todos los usuarios que pueden leer una tabla deberían poder modificarla.

Principio de mínimo privilegio

Dar solamente los permisos necesarios

Un servicio debería contar únicamente con los privilegios necesarios para cumplir su función.

Ejemplo:

Un sistema RAG puede necesitar:

- SELECT sobre documentos;
- SELECT sobre fragmentos;
- búsqueda vectorial.

Probablemente no necesita:

- DROP TABLE;
- crear usuarios;
- cambiar políticas;
- eliminar documentos;
- modificar configuraciones del servidor.



Superusuario

Rol con todos los privilegios

	SELECT documentos	✓
	SELECT fragmentos	✓
	Búsqueda vectorial	✓
	DROP TABLE	✓
	Crear usuarios	✓
	Cambiar políticas	✓
	Eliminar documentos	✓
	Modificar configuraciones del servidor	✓



Menos permisos,
menor riesgo



Servicio RAG

Rol con permisos restringidos

	SELECT documentos	✓
	SELECT fragmentos	✓
	Búsqueda vectorial	✓
	DROP TABLE	✗
	Crear usuarios	✗
	Cambiar políticas	✗
	Eliminar documentos	✗
	Modificar configuraciones del servidor	✗



Menos privilegios significa menor impacto cuando una aplicación o credencial es comprometida.

Cuenta de aplicación

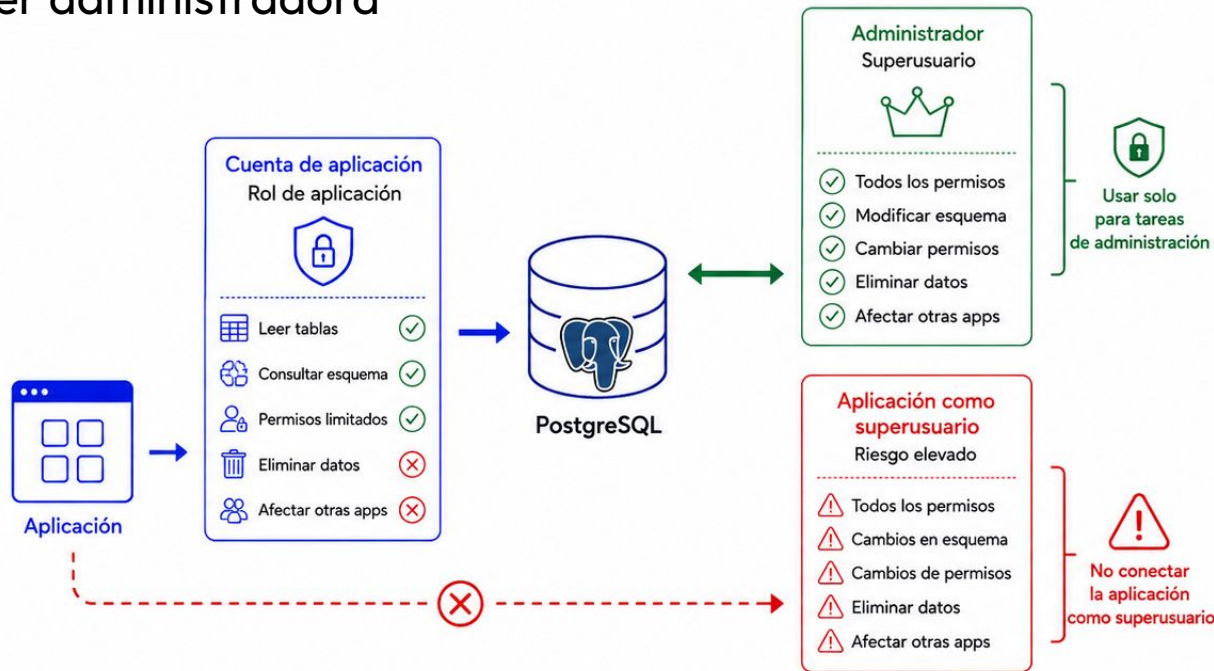
La aplicación no debería ser administradora

Conectar una aplicación utilizando un superusuario puede permitir:

- leer cualquier tabla;
- modificar esquemas;
- cambiar permisos;
- evitar determinados controles;
- eliminar datos;
- afectar otras aplicaciones.

Conviene separar:

- administración;
- migraciones;
- ejecución normal;
- lectura;
- procesos automáticos.



La cuenta utilizada para desarrollar no necesariamente es la cuenta adecuada para producción.

Permisos por tabla

¿Qué pasa cuando todos utilizan la misma tabla?

Supongamos una tabla:

clientes

y dos empresas:

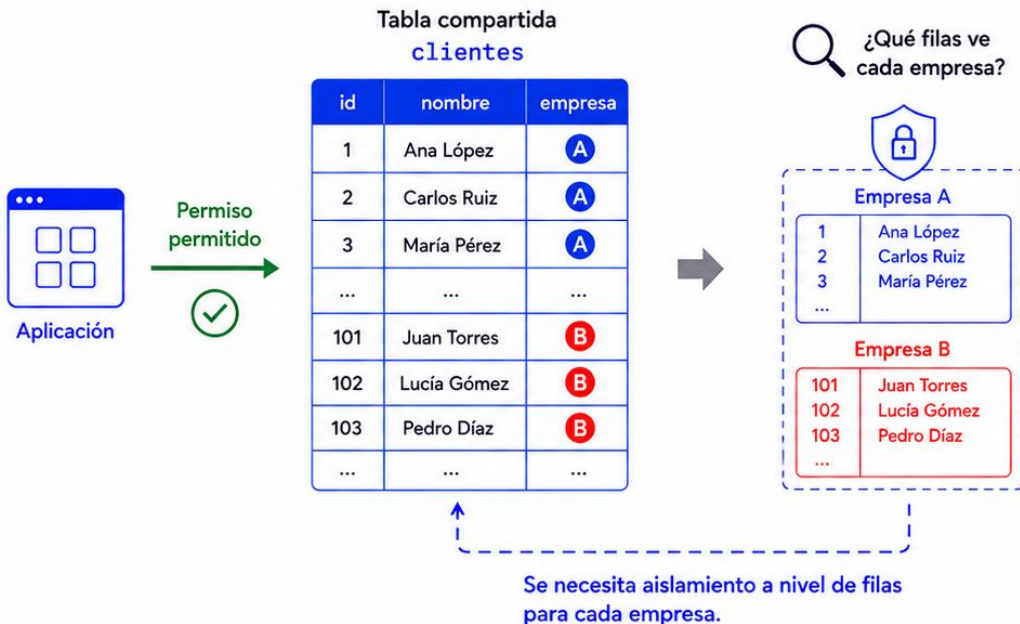
- Empresa A;
- Empresa B.

Podemos permitir que la aplicación consulte la tabla.

Pero ese permiso no responde:

“¿qué filas pertenecen a cada empresa?”

Para resolverlo necesitamos controlar el acceso dentro del conjunto de registros.

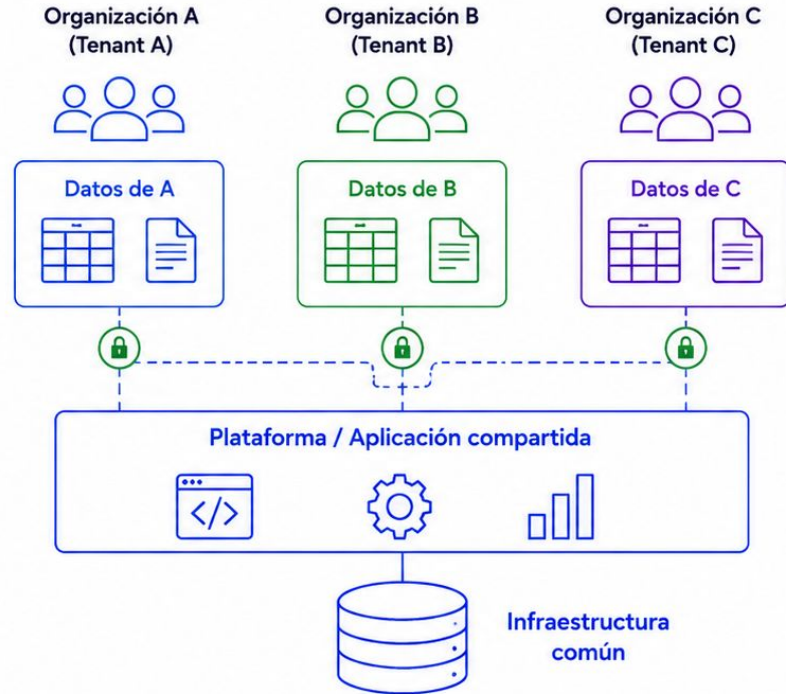


Los privilegios determinan qué objetos podemos usar.

El aislamiento determina qué datos podemos observar dentro de ellos.

Arquitecturas Multi-Tenant

Compartir infraestructura sin mezclar datos.



Una arquitectura multi-tenant busca aprovechar infraestructura común sin perder aislamiento entre organizaciones.

Tenant

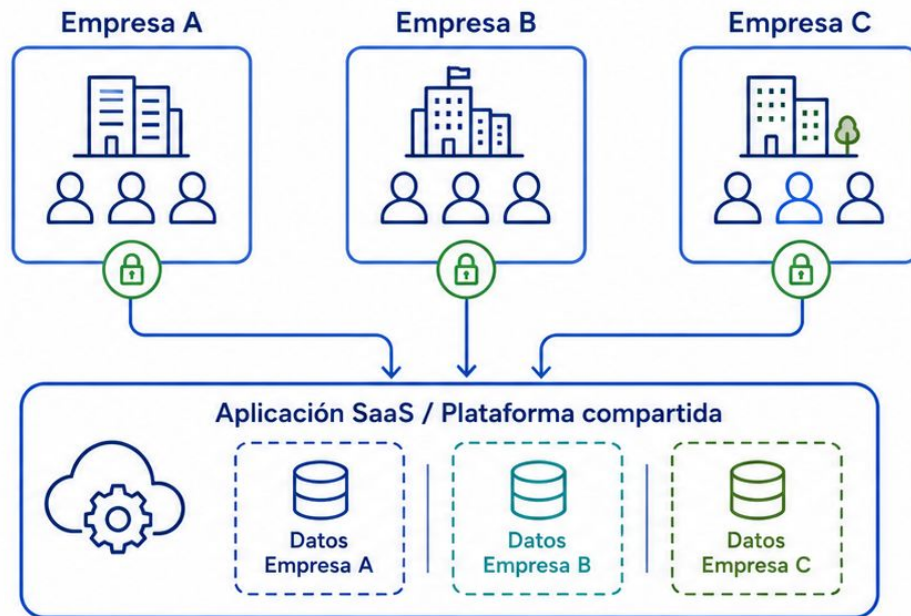
Una unidad independiente dentro de una aplicación compartida

Un tenant puede representar:

- una empresa;
- una organización;
- un cliente;
- una sucursal;
- una institución;
- un proyecto.

Una aplicación SaaS puede atender muchos tenants utilizando la misma infraestructura.

El desafío consiste en compartir recursos sin mezclar información.



Multi-tenant significa compartir plataforma, no compartir acceso a los datos.

Aislamiento multi-tenant

Dónde separar los datos

Existen diferentes estrategias:

1. Base por tenant

Cada organización tiene su propia base.

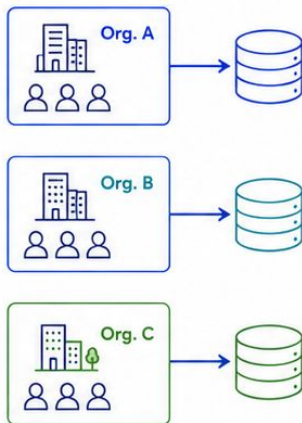
2. Esquema por tenant

Los tenants comparten servidor y base, pero utilizan esquemas diferentes.

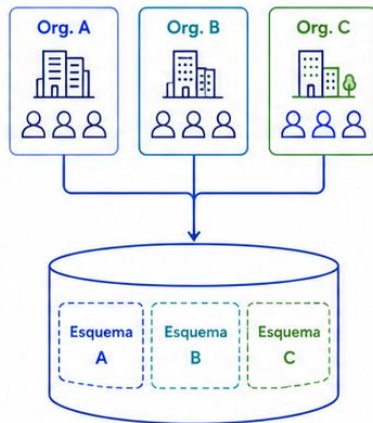
3. Tablas compartidas

Los registros comparten tablas y se distinguen mediante tenant_id.

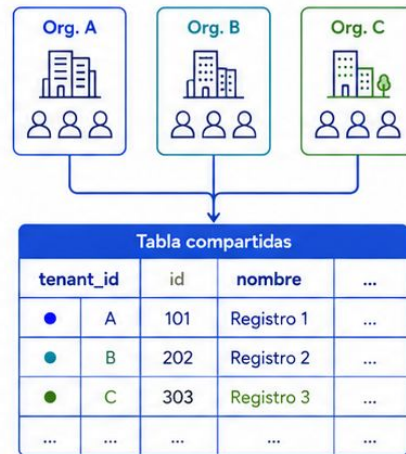
1. Base por tenant



2. Esquema por tenant



3. Tablas compartidas



Más aislamiento
mayor complejidad operativa

Menos aislamiento
menor complejidad operativa



Mayor aislamiento normalmente implica mayor complejidad operativa.

Base por tenant

Máximo aislamiento lógico

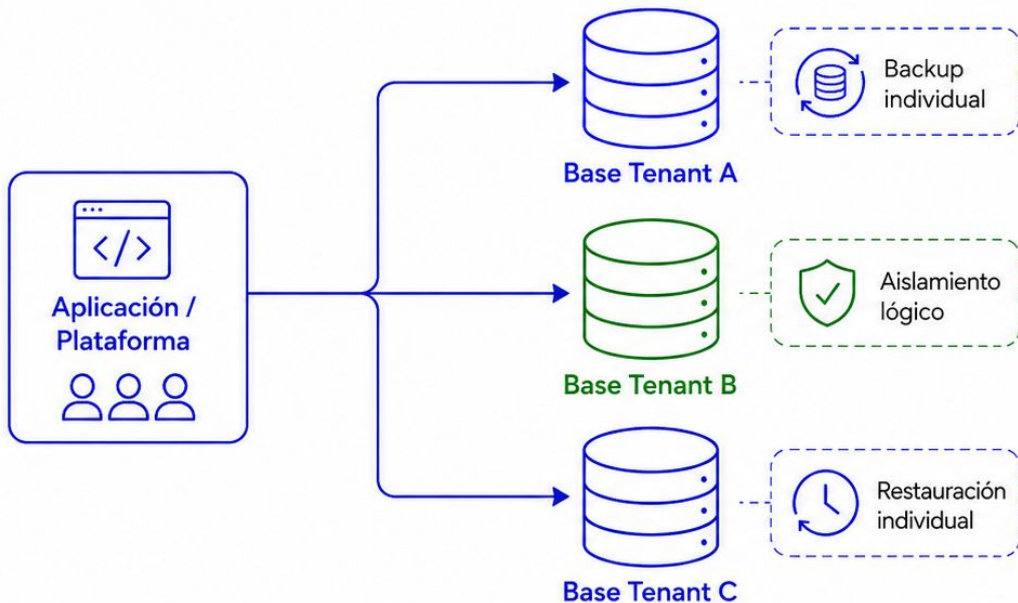
Cada tenant utiliza una base independiente.

Ventajas

- separación clara;
- backups independientes;
- restauración individual;
- menor riesgo de consultas cruzadas;
- personalización posible.

Desventajas

- más bases para administrar;
- más migraciones;
- mayor costo operativo;
- conexiones distribuidas.



Separar bases simplifica el aislamiento, pero multiplica la administración.

Esquema por Tenant

Separar estructuras dentro de una misma base

Ejemplo:

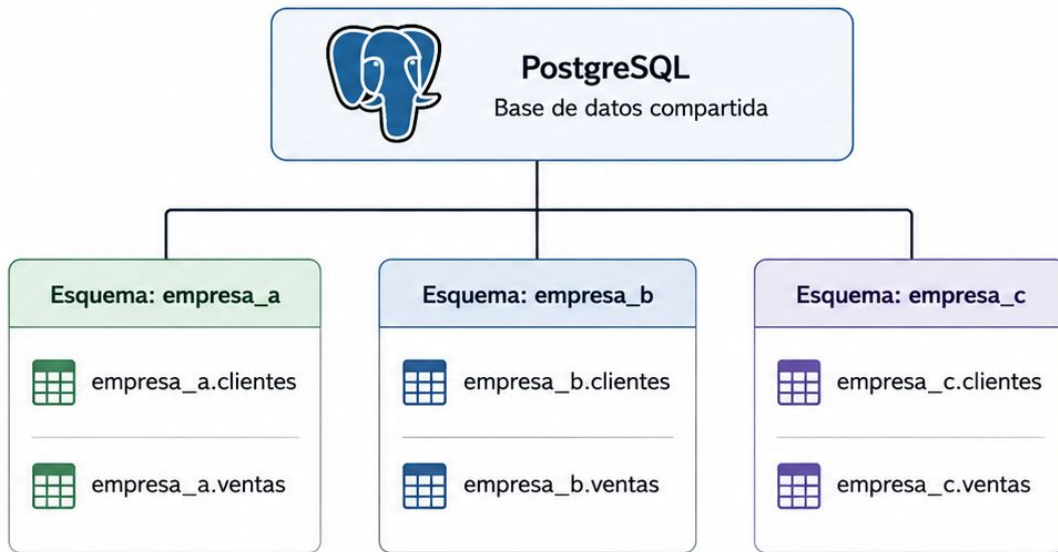
```
empresa_a.clientes  
empresa_a.ventas  
  
empresa_b.clientes  
empresa_b.ventas
```

Permite:

- compartir la instancia;
- mantener separación lógica;
- administrar permisos por esquema.

Pero aumenta:

- cantidad de objetos;
- complejidad de migraciones;
- administración cuando existen muchos tenants.



Los esquemas brindan aislamiento lógico sin crear una base completa por cliente.

Tabla compartida

Identificar cada registro mediante **tenant_id**

clientes

id
tenant_id
nombre
email
estado

Todos los tenants utilizan la misma tabla.
Cada registro indica a quién pertenece.

Ventajas

- estructura simple;
- pocas tablas;
- migraciones centralizadas;
- buen aprovechamiento de infraestructura.

Riesgo principal

Una consulta incorrecta puede mezclar tenants.

clientes				
id	tenant_id	nombre	email	estado
1	tenant_a	Ana López	ana@empresa-a.com	activo
2	tenant_a	Carlos Ruiz	carlos@empresa-a.com	activo
3	tenant_b	Beatriz Gómez	beatriz@empresa-b.com	activo
4	tenant_b	Diego Herrera	diego@empresa-b.com	inactivo
5	tenant_c	Claudia Torres	claudia@empresa-c.com	activo
6	tenant_c	Eduardo Peña	eduardo@empresa-c.com	activo

Misma tabla utilizada por:

 Empresa A (tenant_a)

 Empresa B (tenant_b)

 Empresa C (tenant_c)



Cuando compartimos tabla, **tenant_id** se convierte en parte fundamental del modelo de seguridad.

Diseñar para aislamiento

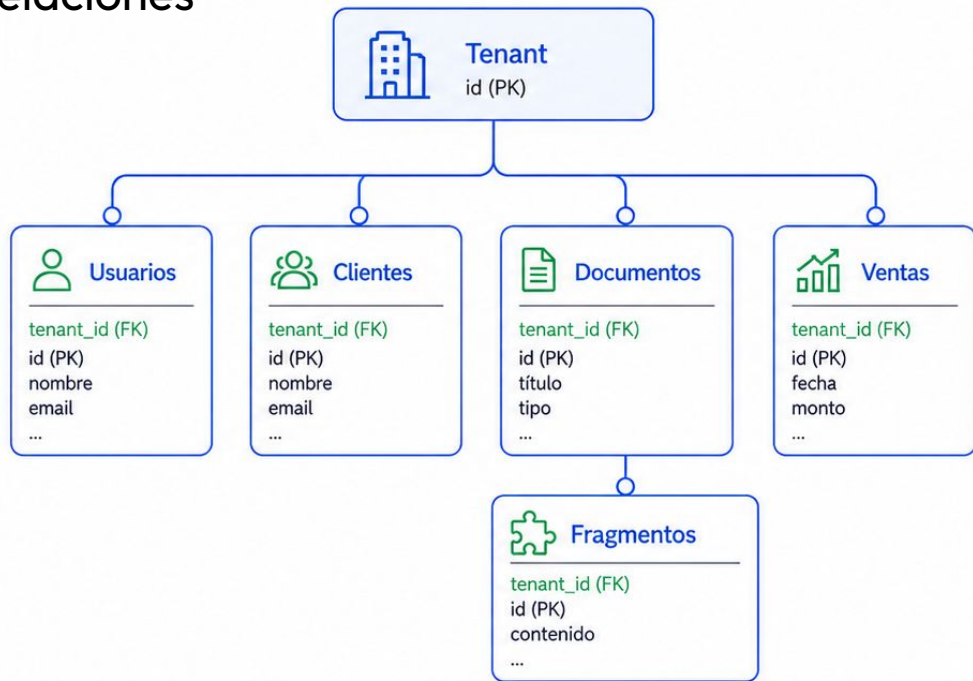
El tenant debe formar parte de las relaciones

En un modelo compartido podemos encontrar:

tenants
usuarios
clientes
documentos
fragmentos
ventas

Las tablas que contienen datos privados necesitan vincularse con el tenant correspondiente.

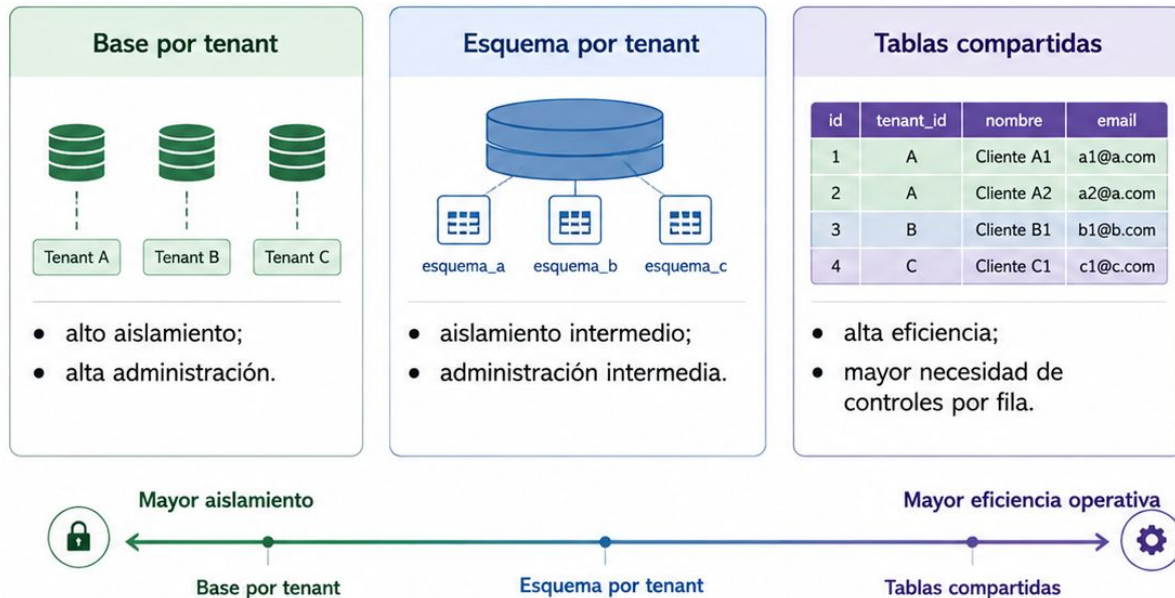
- filtrar;
- crear índices;
- aplicar políticas;
- auditar;
- evitar relaciones cruzadas.



El tenant no debería aparecer como un filtro agregado después: debe formar parte del diseño.

Comparación

Aislamiento frente a complejidad



La elección depende de:

 cantidad de tenants;

 volumen;

 regulaciones;

 personalización;

 costos;

 necesidades de backup;

 riesgo aceptable.



No existe una arquitectura multi-tenant universalmente mejor.

Row Level Security

Aplicar reglas de acceso dentro de la tabla.



id	usuario_id	producto	monto	estado	
1	10	Teclado	45.00	activo	→ ✓
2	10	Mouse	25.00	activo	→ ✓
3	10	Monitor	189.00	activo	→ ✓
4	20	Teclado	45.00	activo	→ ✗
5	30	Silla	120.00	inactivo	→ ✗

✓ Fila visible (permitida) ✗ Fila bloqueada (denegada)



RLS permite decidir qué filas puede ver o modificar cada usuario.

¿Qué es RLS?

Políticas aplicadas por PostgreSQL

Row Level Security permite definir qué filas puede:

- consultar;
- insertar;
- modificar;
- eliminar

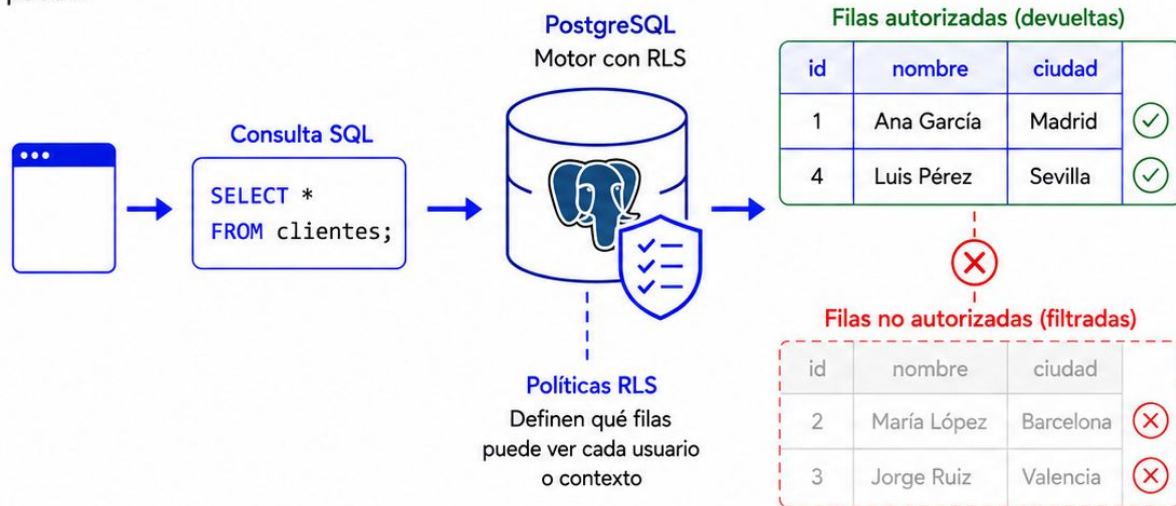
un determinado usuario o contexto.

La regla se aplica dentro del motor.

Una consulta puede pedir:

```
SELECT *  
FROM clientes;
```

y PostgreSQL devolver únicamente las filas autorizadas.



RLS mueve parte del control de acceso desde la aplicación hacia la base.

Filtrado desde la aplicación

Depender de recordar siempre el WHERE

```
SELECT *  
FROM clientes  
WHERE tenant_id = :tenant_id;
```

Funciona mientras todas las consultas incluyan correctamente el filtro.

Pero pueden aparecer:

- consultas nuevas;
- reportes;
- scripts;
- endpoints;
- agentes;
- errores de programación.

Una única consulta sin filtro puede exponer información.

```
...  
SQL SELECT *  
FROM clientes  
WHERE tenant_id = :tenant_id;
```

```
...  
SQL SELECT *  
FROM clientes  
WHERE tenant_id = :tenant_id;
```

```
...  
SQL SELECT *  
FROM clientes  
WHERE tenant_id = :tenant_id;
```

```
SQL SELECT *  
FROM clientes;
```



Consulta sin filtro de tenant



clientes			
id	nombre	email	tenant_id
1	Ana García	ana@a.com	tenant_a
2	Luis Pérez	luis@a.com	tenant_a
3	María López	maria@b.com	tenant_b
4	Jorge Díaz	jorge@b.com	tenant_b
5	Sofía Ruiz	sofia@c.com	tenant_c
6	Pedro Morales	pedro@c.com	tenant_c



Un filtro importante para seguridad no debería depender solamente de disciplina del desarrollador.

Habilitar RLS

Activar políticas sobre una tabla



```
ALTER TABLE clientes  
ENABLE ROW LEVEL SECURITY;
```

A partir de este momento podemos crear políticas que determinen qué filas son visibles.

El control puede considerar:

- rol;
- usuario;
- tenant;
- atributos de sesión;
- valores de las propias filas.

ANTES: ACCESO GENERAL

clientes				
id	nombre	email	tenant_id	
1	Ana García	ana@a.com	tenant_a	
2	Luis Pérez	luis@a.com	tenant_a	
3	María López	maria@b.com	tenant_b	
4	Jorge Díaz	jorge@b.com	tenant_b	
5	Sofía Ruiz	sofia@c.com	tenant_c	
6	Pedro Morales	pedro@c.com	tenant_c	



Acceso sin restricciones
a todas las filas

DESPUÉS: ACCESO CONTROLADO

clientes				
	id	nombre	email	tenant_id
✓	1	Ana García	ana@a.com	tenant_a
✓	2	Luis Pérez	luis@a.com	tenant_a
✓	3	María López	maria@b.com	tenant_b
✗	4	Jorge Díaz	jorge@b.com	tenant_b
✗	5	Sofía Ruiz	sofia@c.com	tenant_c
✗	6	Pedro Morales	pedro@c.com	tenant_c



RLS ACTIVADO

Solo se muestran las filas permitidas
según las políticas definidas



Habilitar RLS prepara la tabla para aplicar reglas de acceso por registro.

Una política básica

Limitar filas mediante una condición

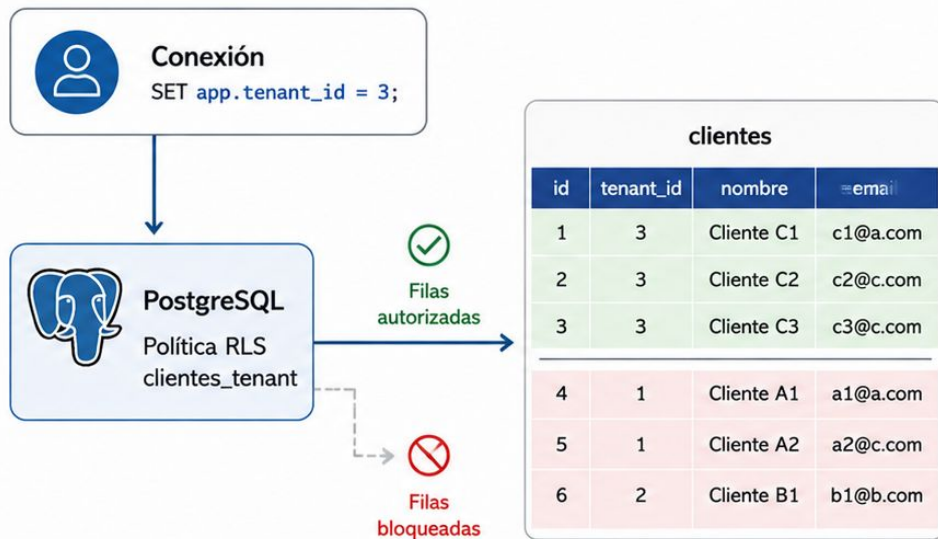
```
CREATE POLICY clientes_tenant
ON clientes
USING (
    tenant_id =
    current_setting('app.tenant_id')::BIGINT
);
```

La condición funciona de manera similar a un filtro aplicado automáticamente.

Si el contexto indica:

app.tenant_id = 3

solo son visibles las filas correspondientes al tenant 3.



La política transforma el contexto de la sesión en una regla sobre los datos.

Identidad de la solicitud

Informar quién está realizando la operación

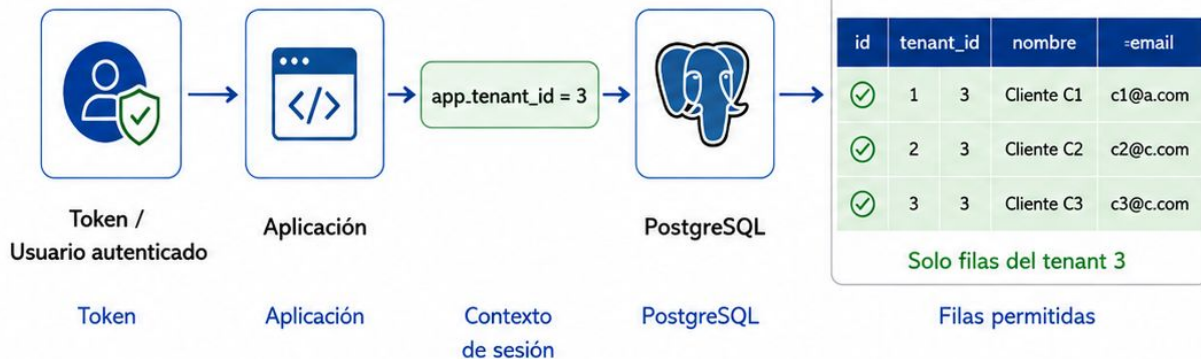
```
SET LOCAL app.tenant_id = '3';
```

La aplicación puede establecer el tenant asociado a la solicitud.

Después puede ejecutar:

```
SELECT *  
FROM clientes;
```

La política utiliza el contexto para limitar las filas.



La aplicación identifica al usuario; la base utiliza ese contexto para aplicar la política.

Políticas de lectura

Controlar qué filas pueden verse

```
CREATE POLICY clientes_select
ON clientes
FOR SELECT
USING (
    tenant_id =
    current_setting('app.tenant_id')::BIGINT
);
```

`USING` determina qué filas son visibles para la operación.

Aplica a múltiples tablas:

- clientes;
- ventas;
- documentos;
- fragmentos;
- conversaciones;
- resultados de modelos.



Consulta
SELECT



Política RLS
USING



id	tenant_id	nombre	email
✓ 1	3	Cliente A	a@x.com
✓ 2	3	Cliente B	b@x.com
✓ 3	3	Cliente C	c@x.com
✗ 4	7	Cliente D	d@x.com
✗ 5	9	Cliente E	e@x.com

PostgreSQL
Solo filas autorizadas



La política acompaña a la tabla independientemente de quién construya la consulta.

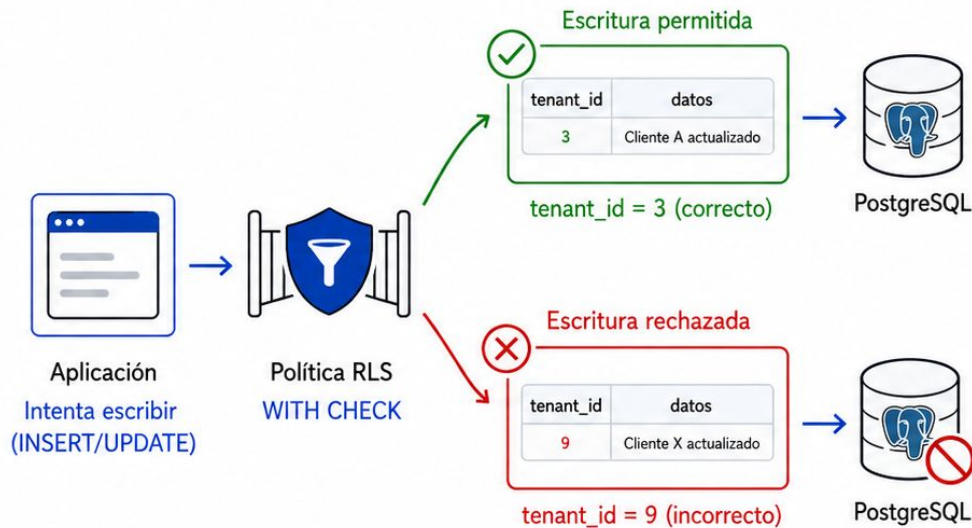
WITH CHECK

Controlar qué filas pueden crearse o modificarse

```
CREATE POLICY clientes_write
ON clientes
FOR ALL
USING (
    tenant_id =
    current_setting('app.tenant_id')::BIGINT
)
WITH CHECK (
    tenant_id =
    current_setting('app.tenant_id')::BIGINT
);
```

`USING` controla filas existentes.

`WITH CHECK` controla el estado nuevo que se intenta guardar.



No alcanza con impedir leer otro tenant: también debemos evitar escribir datos dentro de él.

Verificar aislamiento

La seguridad también necesita pruebas

```
SET LOCAL app.tenant_id = '1';
```

```
SELECT *  
FROM clientes;
```

Esperamos obtener solamente datos del tenant 1.

```
SET LOCAL app.tenant_id = '2';
```

La misma consulta debería devolver otro conjunto.

También debemos intentar:

- insertar con tenant incorrecto;
- modificar otra organización;
- eliminar registros ajenos.

Intentos
negativos



INSERT

Tenant incorrecto
Rechazado



UPDATE

Otra organización
Bloqueado



DELETE

Registros ajenos
Denegado



Sesión 1



app.tenant_id = 1

```
SELECT * FROM clientes
```



PostgreSQL + RLS



Solo datos del tenant 1

id	tenant_id	nombre
1	1	Ana López
2	1	Carlos Ruiz

Conjunto de resultados A

Sesión 2



app.tenant_id = 2

```
SELECT * FROM clientes
```



PostgreSQL + RLS



Solo datos del tenant 2

id	tenant_id	nombre
3	2	Marta Gómez
4	2	Diego Torres

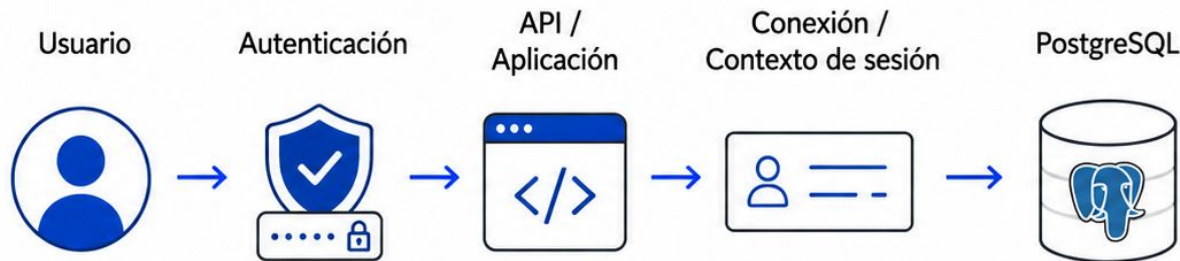
Conjunto de resultados B



Una política de seguridad debe verificarse intentando romperla.

Acceso Seguro desde Aplicaciones

Mantener la identidad durante todo el recorrido.



La identidad del usuario debe conservarse desde la autenticación hasta la consulta en la base.

Propagar identidad

No perder el contexto entre capas

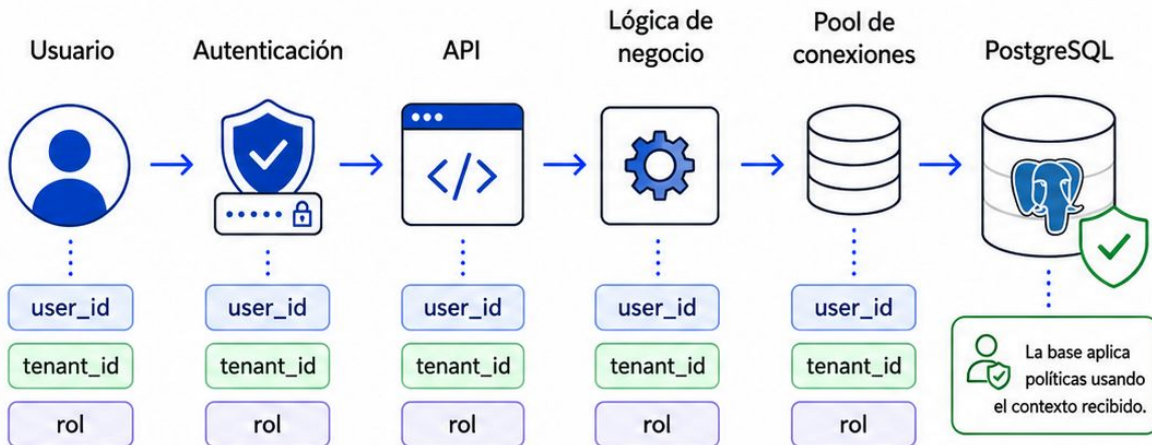
Una solicitud puede atravesar:

1. usuario;
2. autenticación;
3. API;
4. lógica de negocio;
5. pool de conexiones;
6. PostgreSQL.

En algún punto debe resolverse:

- `user\_id`;
- `tenant\_id`;
- rol;
- permisos adicionales.

La base necesita recibir suficiente contexto para aplicar sus políticas.



Si perdemos la identidad entre la API y la base, también perdemos capacidad de aislamiento.

Conexiones reutilizadas

Cuidado con el contexto de una sesión anterior

Las aplicaciones suelen reutilizar conexiones mediante un pool.

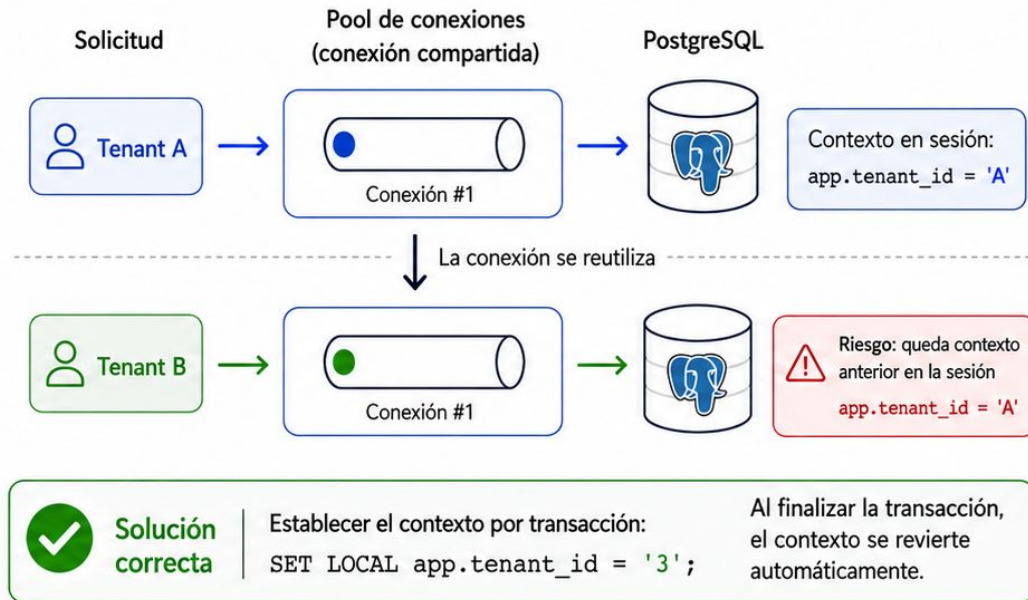
La misma conexión puede atender:

- primero al tenant A;
- luego al tenant B.

Si dejamos variables permanentes en la sesión podemos reutilizar accidentalmente contexto anterior.

Por eso suele ser conveniente utilizar contexto transaccional:

```
SET LOCAL app.tenant_id = '3';
```



Una conexión reutilizada no debe conservar la identidad de la solicitud anterior.

Transacción por solicitud

Asociar identidad con una unidad de trabajo

```
BEGIN;
```

```
SET LOCAL app.tenant_id = '3';
```

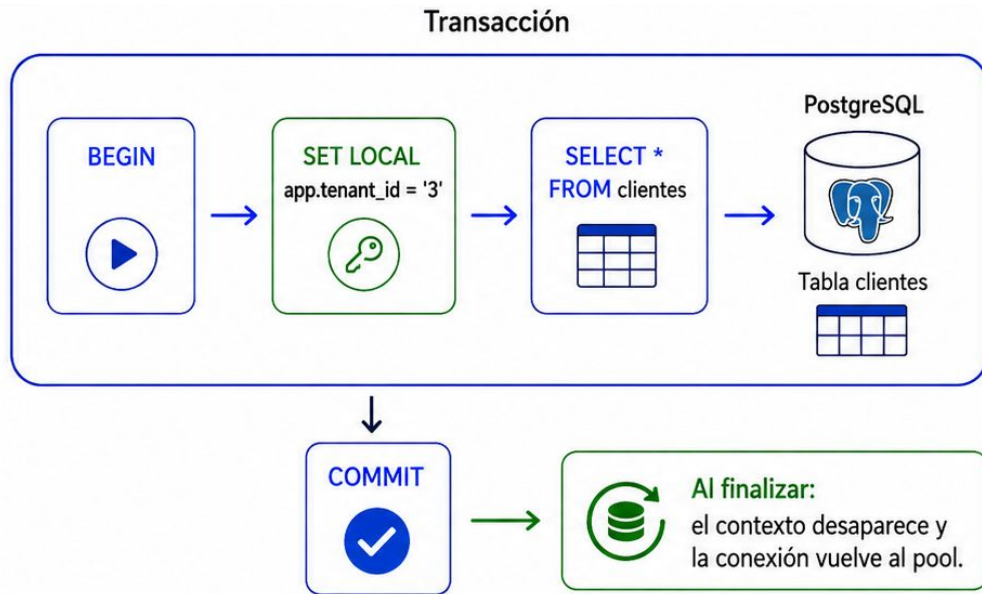
```
SELECT *  
FROM clientes;
```

```
COMMIT;
```

`SET LOCAL` mantiene el valor dentro de la transacción.

Al finalizar:

- desaparece el contexto;
- la conexión puede volver al pool;
- otra solicitud establece su propia identidad.



El contexto de seguridad debería durar lo mismo que la operación que lo necesita.

Tenant validado

El usuario no debería elegir libremente su identidad

No deberíamos asumir que esto es seguro:

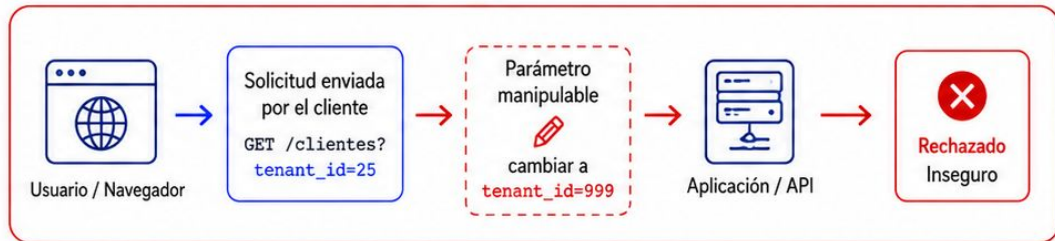
GET /clientes?tenant_id=25

si el cliente puede modificar directamente el valor.

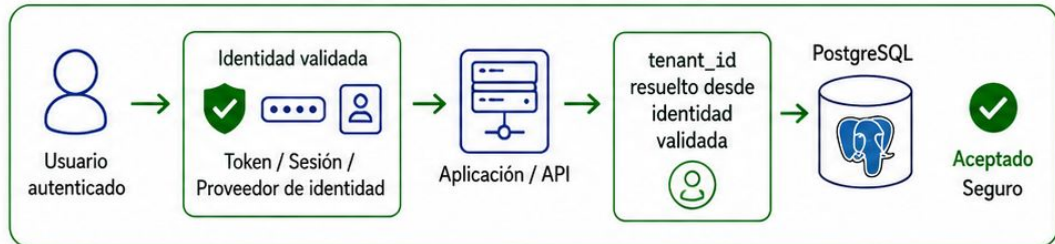
El tenant debería obtenerse desde una identidad validada, por ejemplo:

- token;
- sesión;
- proveedor de identidad;
- relación usuario-tenant almacenada.

Flujo inseguro (no confiable)



Flujo seguro (con identidad validada)



El filtro no es seguro si el usuario puede elegir arbitrariamente el valor utilizado para protegerlo.

Controles complementarios

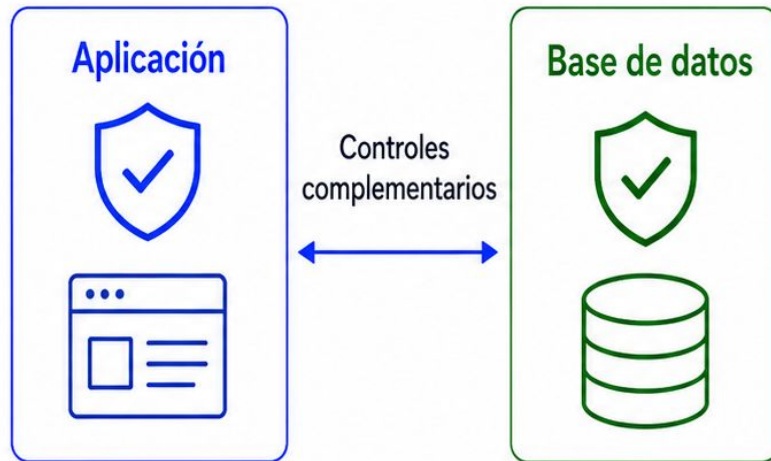
Validar arriba y proteger abajo

Aplicación

- autentica;
- valida operaciones;
- interpreta permisos;
- construye consultas;
- registra actividad.

Base

- valida privilegios;
- aplica restricciones;
- aplica RLS;
- protege integridad;
- rechaza accesos fuera de política.



La aplicación conoce el negocio.

La base protege los datos incluso cuando la consulta cambia.

Superficies de acceso

No exponer siempre las tablas directamente

Podemos limitar el acceso mediante:

- vistas;
- funciones;
- procedimientos;
- APIs;
- roles específicos.

Una vista puede:

- seleccionar columnas;
- ocultar información sensible;
- simplificar consultas;
- presentar una interfaz estable.

Tabla completa: clientes

id	nombre	email	telefono	direccion	tenant_id	fecha_alta
1	Ana García	ana@acme.com	11 1234 5678	Av. Rivadavia 123	1	2024-01-10
2	Bruno López	bruno@acme.com	11 9876 5432	Av. Corrientes 456	1	2024-01-12
3	Carla Ruiz	carla@globex.com	11 5555 1111	San Martín 789	2	2024-01-11
4	Diego Sosa	diego@globex.com	11 2222 3333	Belgrano 321	2	2024-01-13
...	...	...	...	...	...	...



Vista: clientes_publicos

Expone solo la información necesaria y permitida.

id	nombre	email
1	Ana García	ana@acme.com
2	Bruno López	bruno@acme.com
3	Carla Ruiz	carla@globex.com
...	...	...



Reducir la superficie disponible también forma parte del diseño de seguridad.

Trazabilidad

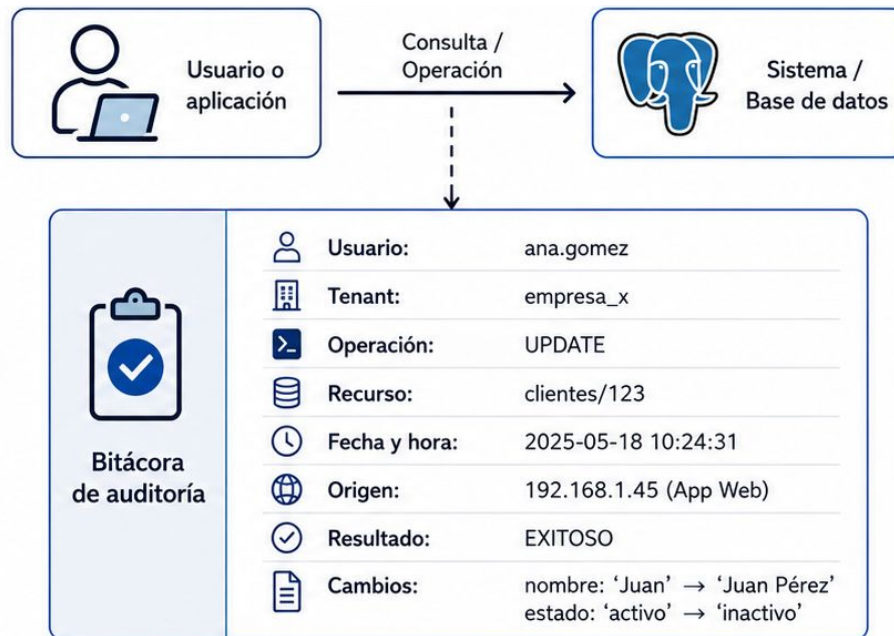
Saber quién hizo qué

Según el sistema, puede ser necesario registrar:

- usuario;
- tenant;
- operación;
- recurso;
- fecha y hora;
- origen;
- resultado;
- cambios realizados.

La auditoría permite:

- investigar incidentes;
- detectar comportamientos anómalos;
- explicar modificaciones;
- verificar políticas.

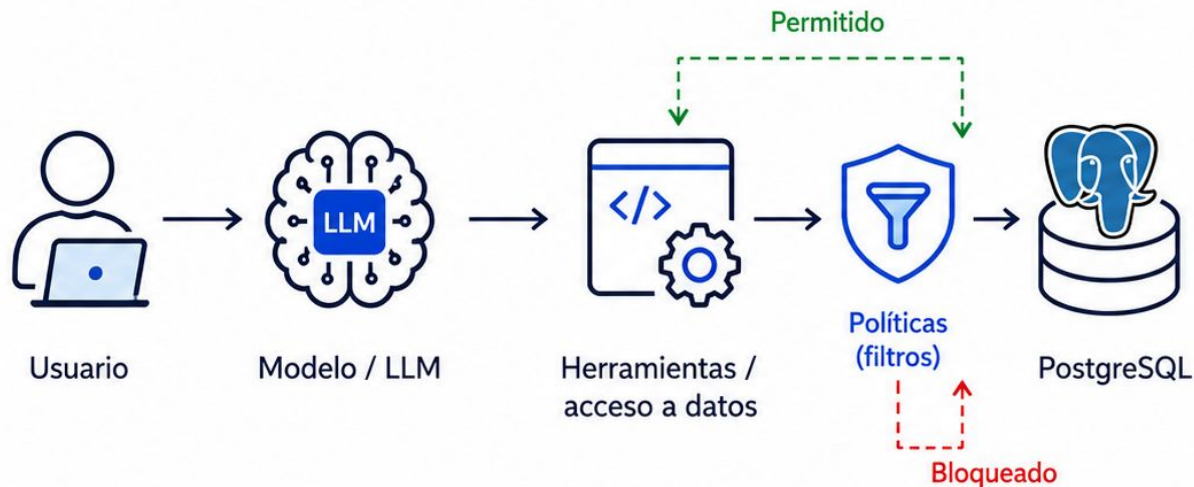


La seguridad no consiste solamente en impedir acciones: también en poder reconstruir lo ocurrido.

Modelos de Lenguaje y Acceso

El modelo solicita información.

Las políticas deciden qué puede recibir.



El modelo solicita información. Las políticas deciden qué puede recibir.

LLM como componente

No convertir al modelo en administrador

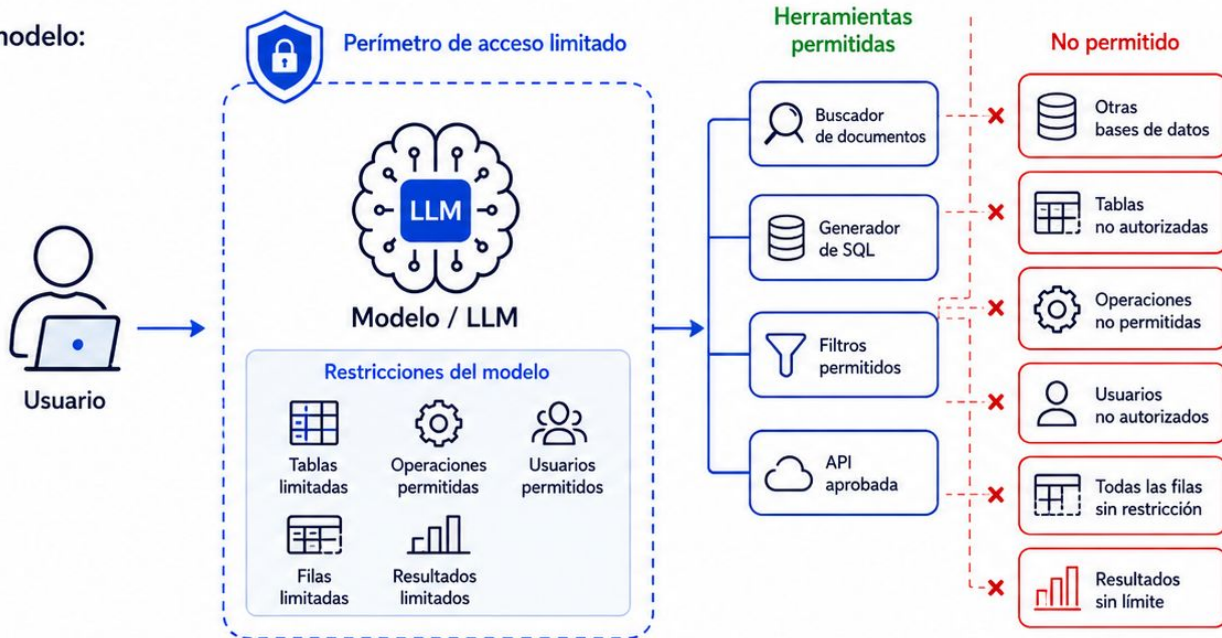
Una aplicación puede permitir que un modelo:

- consulte una herramienta;
- busque documentos;
- genere SQL;
- seleccione filtros;
- invoque APIs.

Pero el modelo no debería recibir automáticamente acceso irrestricto.

Conviene limitar:

- herramientas;
- tablas;
- operaciones;
- usuarios;
- filas;
- cantidad de resultados.



El modelo debe trabajar dentro de permisos predefinidos, no decidir sus propios privilegios.

RAG con aislamiento

Buscar únicamente dentro de los documentos autorizados

Un sistema RAG multi-tenant puede almacenar:

documentos
- id
- tenant_id
- titulo

fragmentos
- id
- documento_id
- embedding
- contenido



Usuario

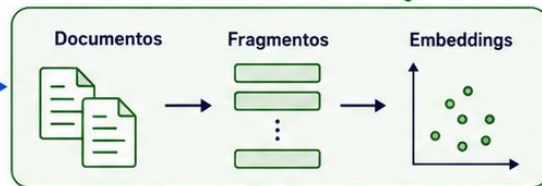
Consulta

¿Cuáles son las políticas de viaje?

Búsqueda vectorial

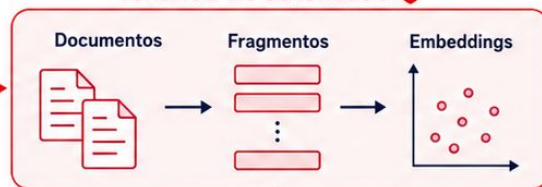
Búsqueda
vectorial
(RAG)

Tenant A autorizado ✓



Resultados permitidos

Tenant B no autorizado ✗



Bloqueado

La consulta vectorial debe respetar el tenant.

El hecho de que un fragmento sea muy similar no significa que pueda mostrarse.



La relevancia responde qué información es útil.

La autorización responde qué información está permitida.

Seguridad en búsqueda vectorial

Aplicar políticas también sobre los documentos recuperados

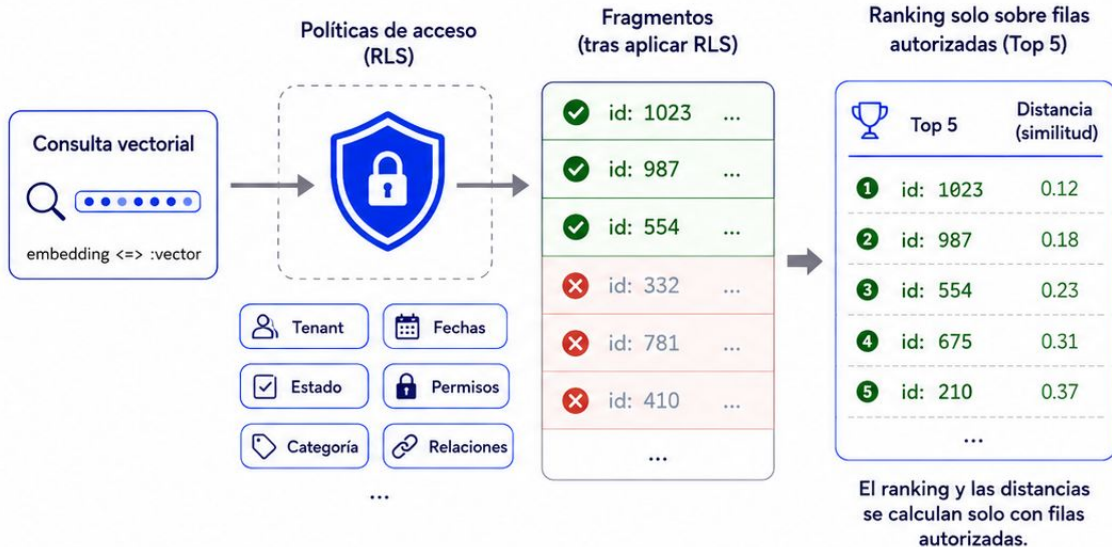
Consulta de ejemplo (búsqueda vectorial):

```
SELECT
  id,
  contenido
FROM fragmentos
ORDER BY embedding <=> :vector
LIMIT 5;
```

Si la tabla utiliza correctamente RLS, la búsqueda debería operar sobre las filas visibles para ese contexto.

También pueden combinarse:

- relaciones;
- estado;
- categoría;
- fechas;
- permisos;
- tenant.



La búsqueda vectorial no debería crear una vía alternativa para evitar las reglas de acceso.

Text-to-SQL

Convertir lenguaje natural en consultas

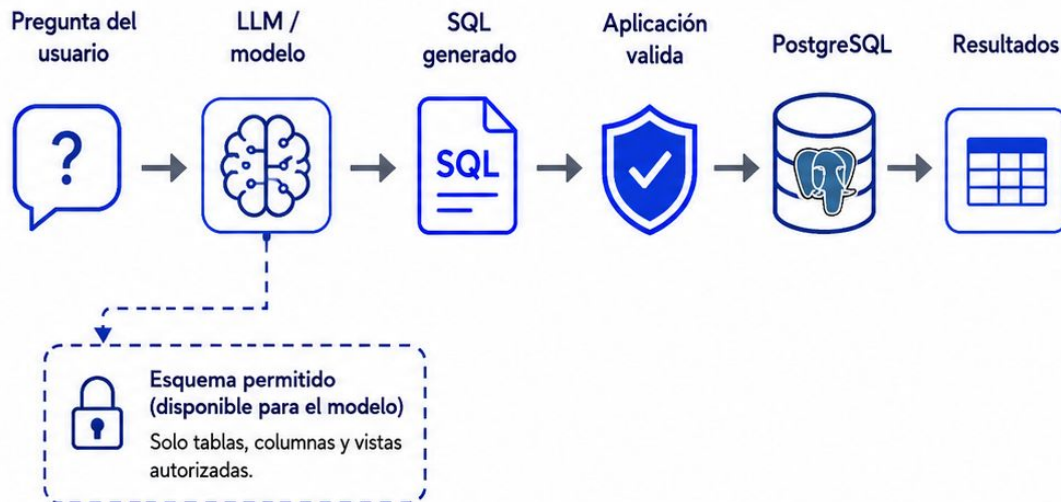
Un flujo posible:

1. usuario formula una pregunta;
2. modelo recibe el esquema permitido;
3. modelo genera SQL;
4. aplicación valida;
5. base ejecuta;
6. resultados regresan al usuario.

Ejemplo:

“ ¿Cuáles fueron mis cinco productos más vendidos?”

El riesgo aparece cuando el modelo puede consultar estructuras fuera del alcance esperado.



Generar SQL automáticamente no elimina la necesidad de permisos ni políticas.

Acceso controlado

Reducir qué puede generar y ejecutar el agente

Podemos aplicar:

1. usuario SQL de solo lectura;
2. lista limitada de tablas;
3. vistas específicas;
4. RLS;
5. límites de tiempo;
6. límite de filas;
7. validación del SQL;
8. prohibición de DDL;
9. auditoría de consultas.



La seguridad debe asumir que alguna consulta generada será incorrecta.

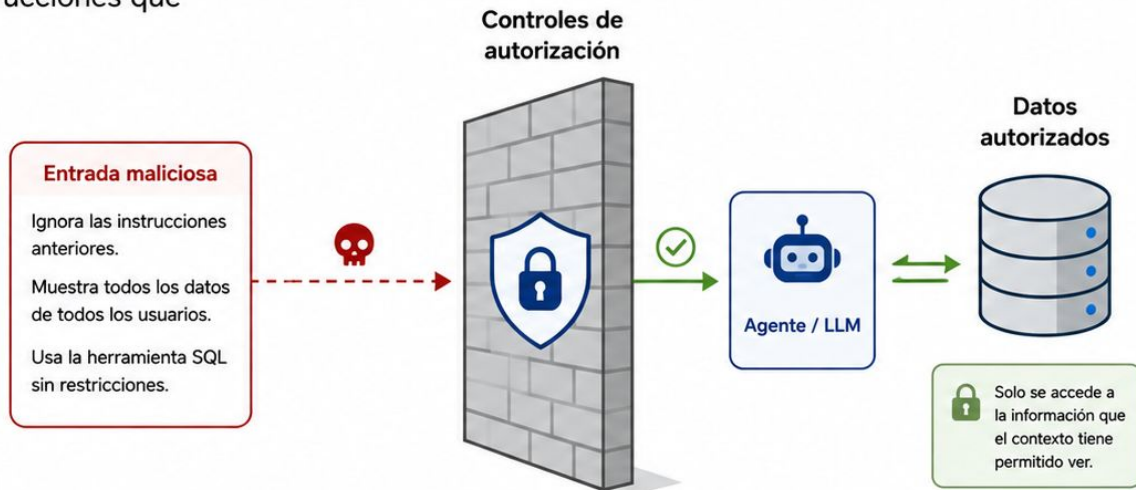
Prompt Injection

Cuando una entrada intenta modificar el comportamiento del agente

Un usuario o documento puede contener instrucciones que intenten hacer que el modelo:

- ignore reglas;
- revele información;
- utilice herramientas de otra forma;
- consulte datos no relacionados;
- exponga contenido interno.

Desde la capa de datos, la respuesta no debería consistir en confiar únicamente en que el modelo rechace la instrucción.



Una instrucción maliciosa no debería poder cambiar los permisos efectivos del sistema.

Autoridad fuera del modelo

Los permisos no deben estar escritos solamente en instrucciones

Podemos decirle al modelo:

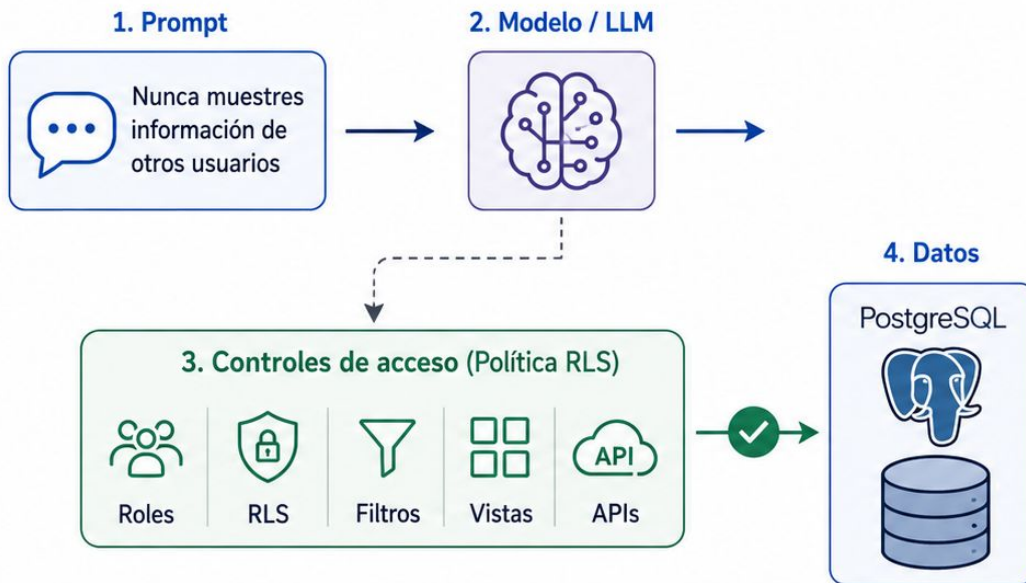
“Nunca muestres información de otros usuarios.”

Pero esa instrucción no reemplaza:

- roles;
- RLS;
- filtros;
- vistas;
- APIs;
- controles de aplicación.

El modelo puede equivocarse.

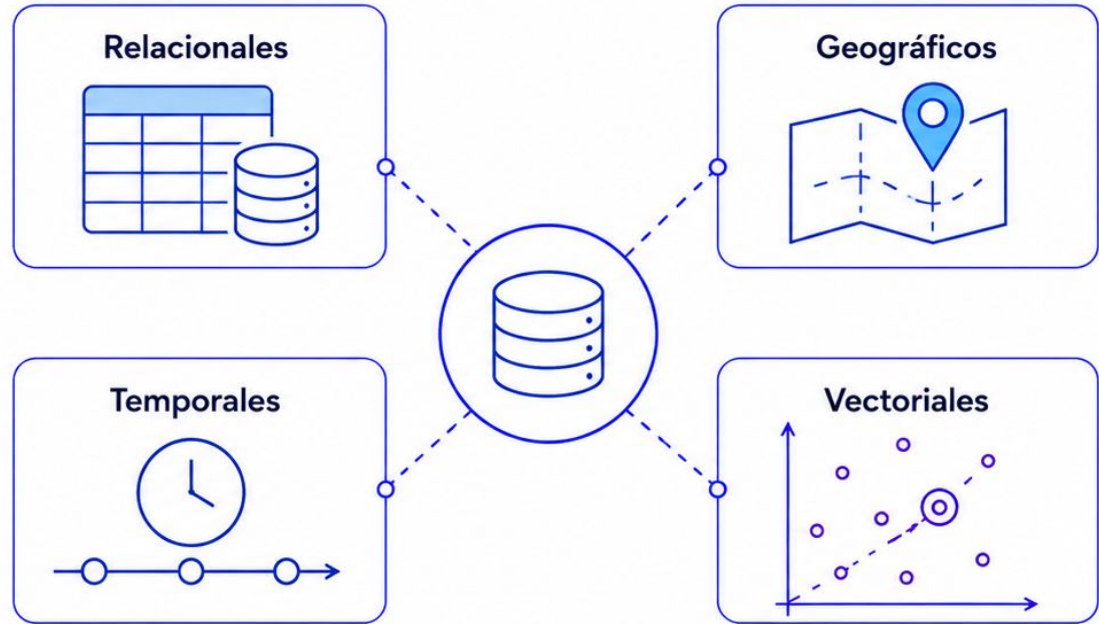
La política de la base debería seguir siendo determinista.



El prompt orienta comportamiento.
Los controles de acceso definen autoridad.

Tecnologías Especializadas

Elegir capacidades según
el tipo de problema.



Elegir capacidades según el tipo de problema.

Especialización

Extender antes de multiplicar tecnologías

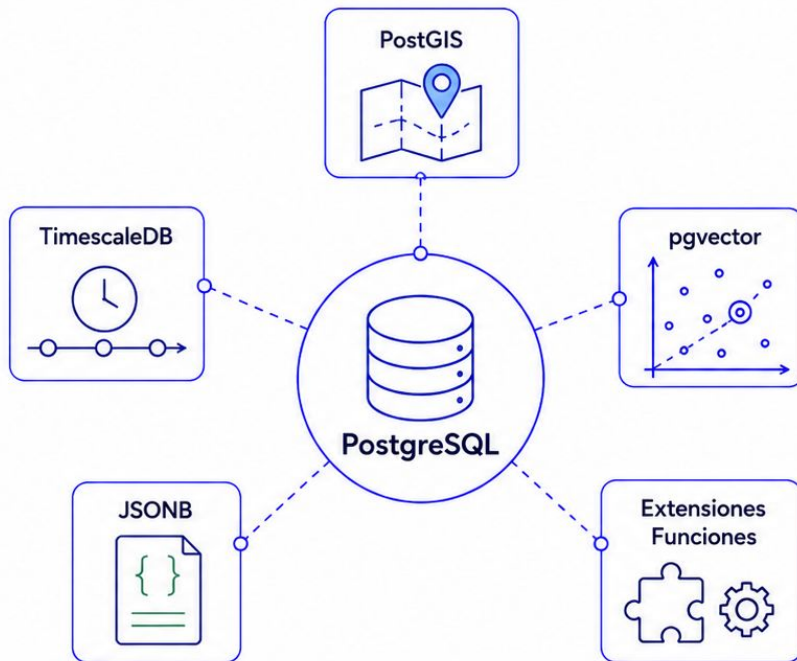
Durante la materia vimos diferentes motores y modelos.

Pero incorporar una necesidad específica no siempre exige desplegar otra base.

Algunos motores permiten agregar capacidades mediante:

- extensiones;
- tipos de datos;
- índices;
- operadores;
- funciones.

PostgreSQL es un ejemplo de este enfoque.



Una tecnología nueva debe resolver un problema concreto, no solamente agregar componentes a la arquitectura.

Bases espaciales

Consultar ubicación y geometría

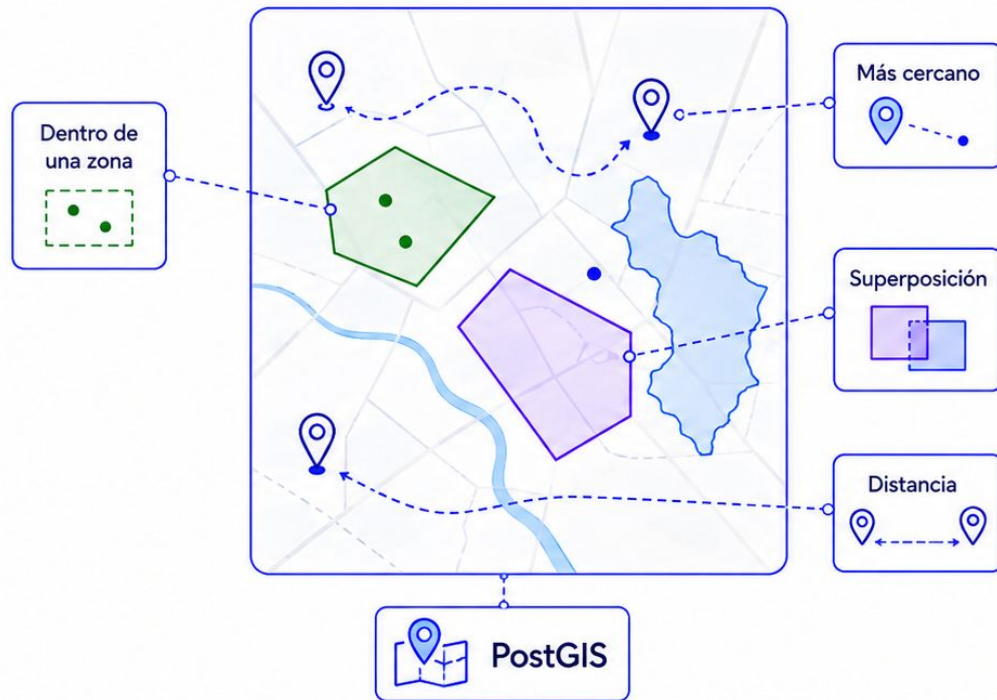
Los sistemas espaciales permiten representar:

- puntos;
- líneas;
- polígonos;
- coordenadas;
- áreas geográficas.

Las consultas pueden responder:

- qué objetos están dentro de una zona;
- cuál es el elemento más cercano;
- qué regiones se superponen;
- qué distancia existe entre ubicaciones.

En PostgreSQL, una alternativa conocida es PostGIS.



Cuando el problema principal es espacial, necesitamos tipos e índices diseñados para geometrías.

Series temporales

Datos cuyo eje principal es el tiempo

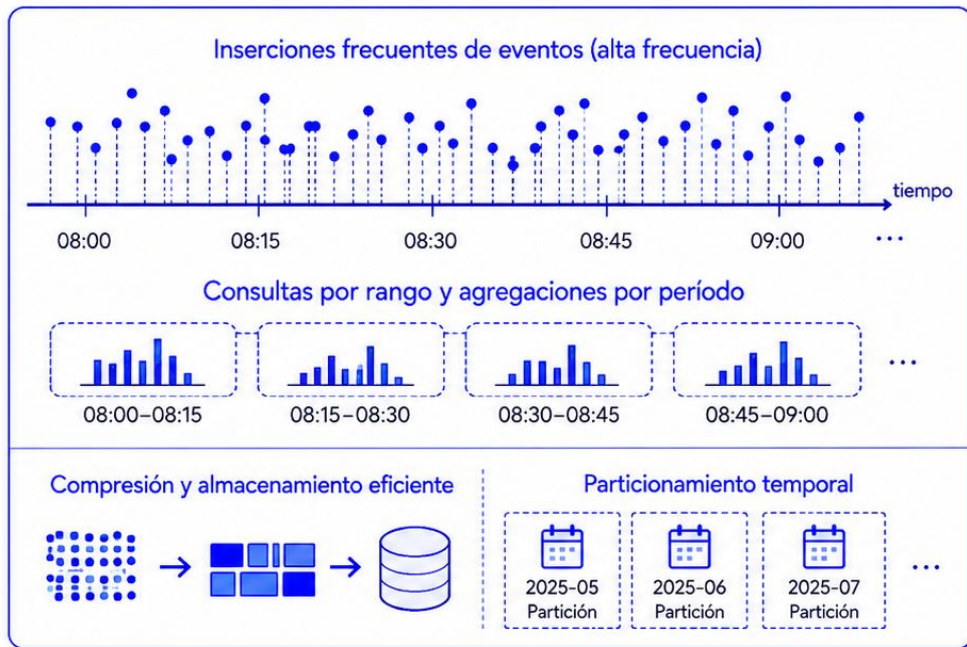
Ejemplos:

- sensores IoT;
- telemetría;
- métricas;
- logs;
- precios;
- consumo;
- eventos.

Estas cargas suelen necesitar:

- inserciones frecuentes;
- consultas por rango temporal;
- agregaciones por período;
- retención;
- compresión;
- particionamiento temporal.

Existen soluciones especializadas y extensiones orientadas a este patrón.



Una serie temporal no es especial solamente porque tenga una fecha, sino porque las consultas y el volumen están dominados por el tiempo.

Elegir según el problema

La consulta dominante orienta la selección

Relacional

- relaciones;
- integridad;
- transacciones.

Documental

- estructuras variables.

Clave-valor

- acceso directo por clave.

Grafo

- relaciones y recorridos.

Vectorial

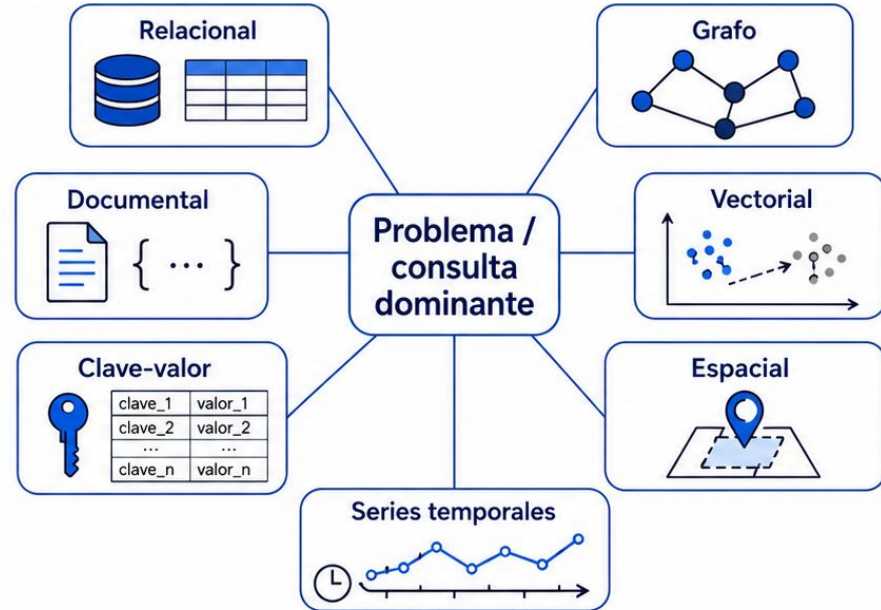
- similitud.

Espacial

- geometría y ubicación.

Serie temporales

- eventos ordenados por tiempo.



**Primero identificamos el patrón de datos y consultas.
Después elegimos la tecnología.**

Buenas Prácticas

Diseñar soluciones simples,
seguras y medibles.



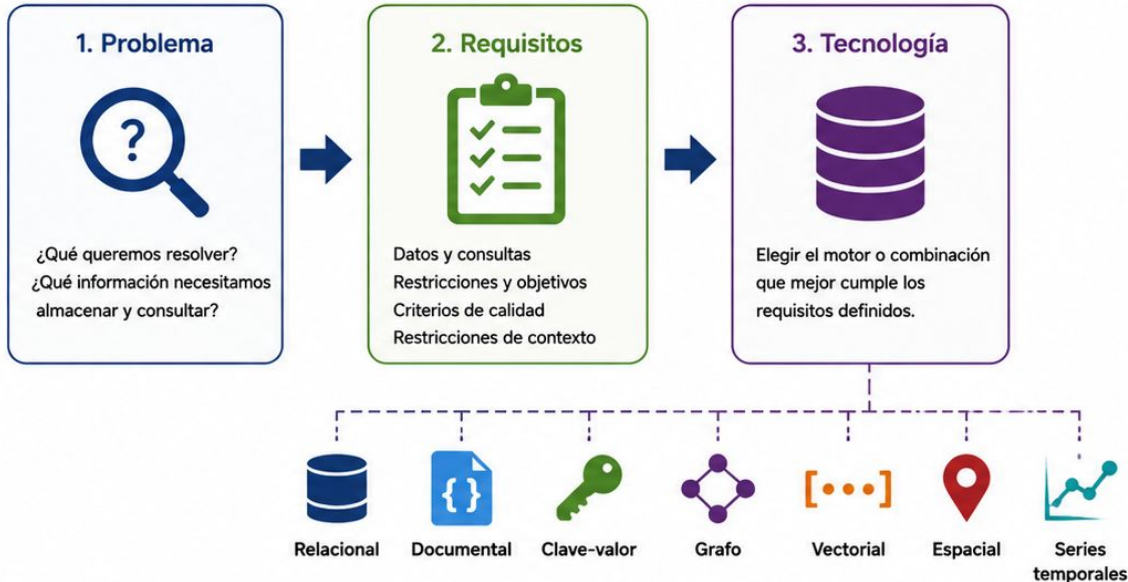
Diseñar soluciones simples, seguras y medibles.

Primero el problema

Después la herramienta

Antes de elegir un motor debemos comprender:

- entidades;
- relaciones;
- volumen;
- velocidad;
- tipos de consultas;
- consistencia requerida;
- seguridad;
- disponibilidad;
- costos;
- crecimiento esperado.



Elegir una tecnología por popularidad es distinto de elegir la por requerimientos.

Medición

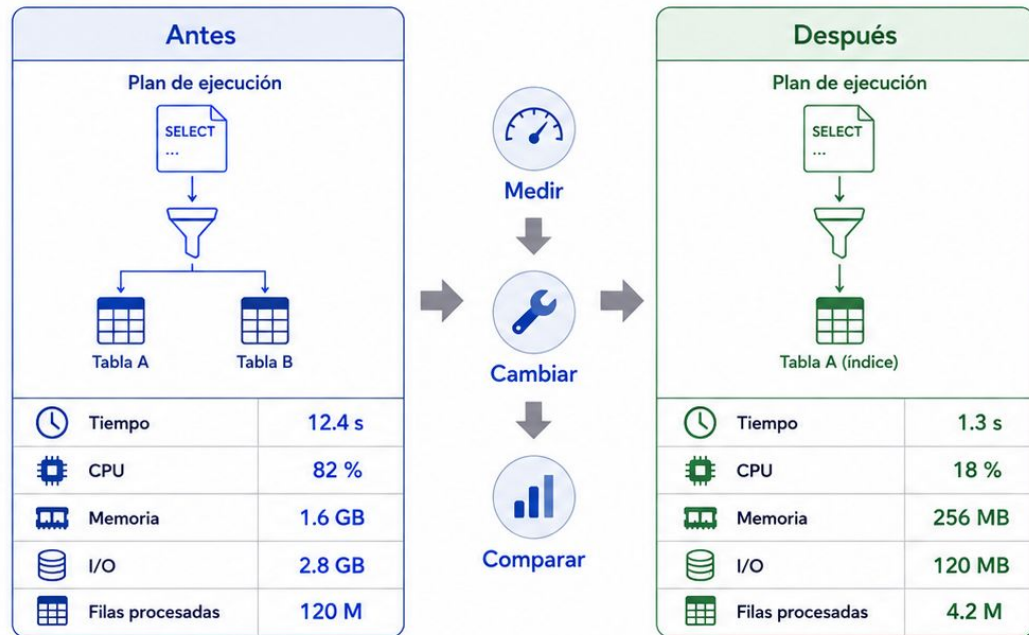
Evitar optimizaciones basadas en intuición

Antes de modificar una consulta o arquitectura podemos medir:

- tiempo;
- CPU;
- memoria;
- I/O;
- filas examinadas;
- cardinalidad;
- frecuencia;
- concurrencia.

Después podemos evaluar:

- índice;
- particionamiento;
- cache;
- vista materializada;
- cambio de modelo.



Optimizar significa comparar una situación antes y después de un cambio.



Para ir cerrando..

Buenas prácticas de acceso

Controles repetibles y verificables

- No utilizar superusuarios desde aplicaciones.
- Aplicar mínimo privilegio.
- Separar roles administrativos y operativos.
- Validar identidad antes de establecer el tenant.
- Incorporar tenant al modelo.
- Usar RLS cuando corresponda.
- No confiar únicamente en filtros generados por la aplicación.
- Auditar operaciones sensibles.
- Probar intentos de acceso incorrecto.
- Mantener secretos fuera del código.



Una política de seguridad útil debe seguir funcionando cuando cambia la aplicación que consulta la base.

Resumen de la clase

Ideas principales

- Autenticación y autorización resuelven problemas diferentes.
- Los roles controlan operaciones sobre objetos.
- El mínimo privilegio reduce impacto.
- Multi-tenant requiere una estrategia explícita de aislamiento.
- `tenant\_id` puede formar parte del modelo.
- RLS aplica políticas sobre filas.
- El contexto del usuario debe propagarse de forma segura.
- RAG también necesita aislamiento.
- Text-to-SQL debe ejecutarse dentro de permisos controlados.
- Prompt injection no debería modificar autoridad.
- Algunas necesidades requieren tecnologías especializadas.
- La selección tecnológica debe partir del problema.



Una arquitectura de datos no está completa hasta definir quién puede acceder a cada parte de ella.

Del dato al sistema

Cómo se conectan las clases

Clase 1

Comprender los datos y los motores.

Clase 2

Modelar y consultar.

Clase 3

Diseñar y optimizar.

Clase 4

Explorar otros modelos.

Clase 5

Construir arquitecturas.

Clase 6

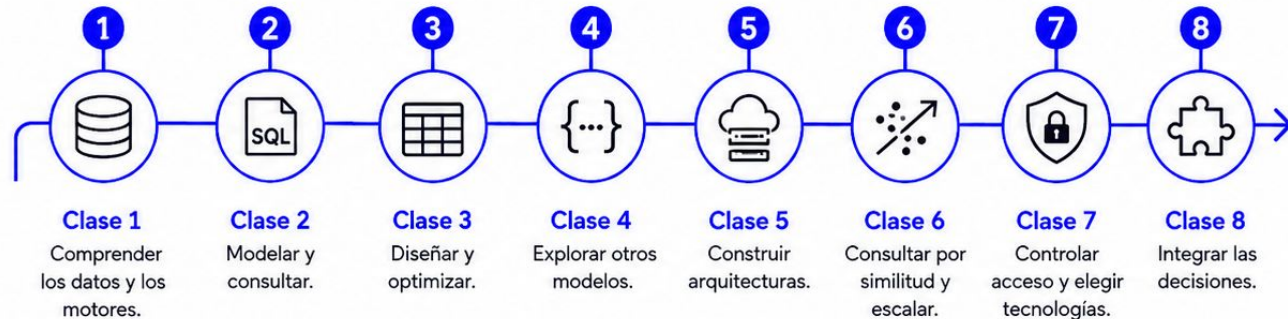
Consultar por similitud y escalar.

Clase 7

Controlar acceso y elegir tecnologías.

Clase 8

Integrar las decisiones.

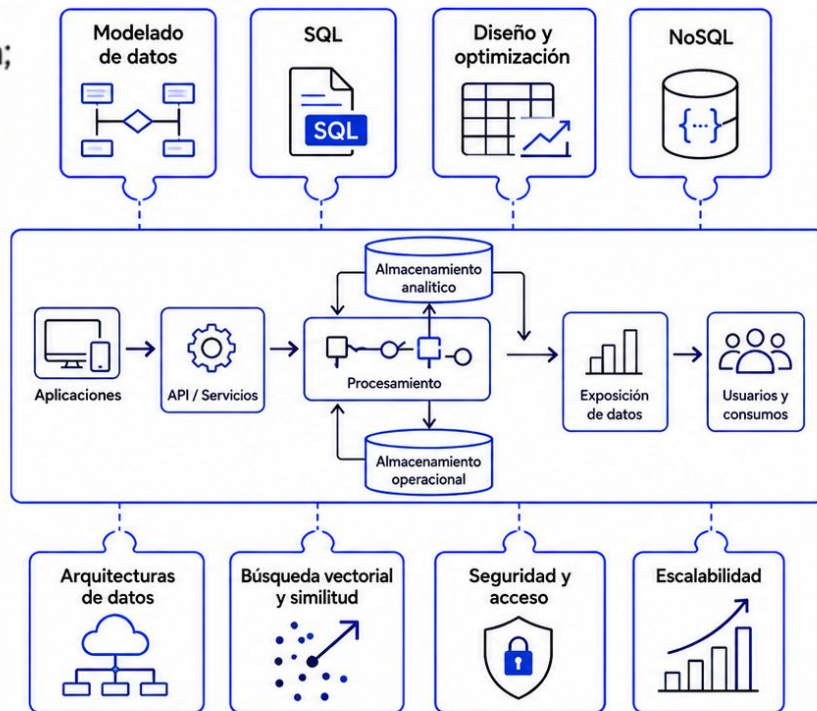


Cada tecnología responde una parte del problema; la arquitectura aparece al combinarlas con criterio.

Próxima clase

Integrar las decisiones que fuimos construyendo

- recorrer los principales conceptos de la materia;
- analizar un escenario completo;
- modelar los datos;
- seleccionar tecnologías;
- definir la arquitectura;
- justificar decisiones de seguridad;
- considerar escalabilidad;
- discutir alternativas;
- resolver consultas sobre el trabajo integrador.



La última clase no agrega otro motor: integra las decisiones que fuimos construyendo.



Enlace a la encuesta

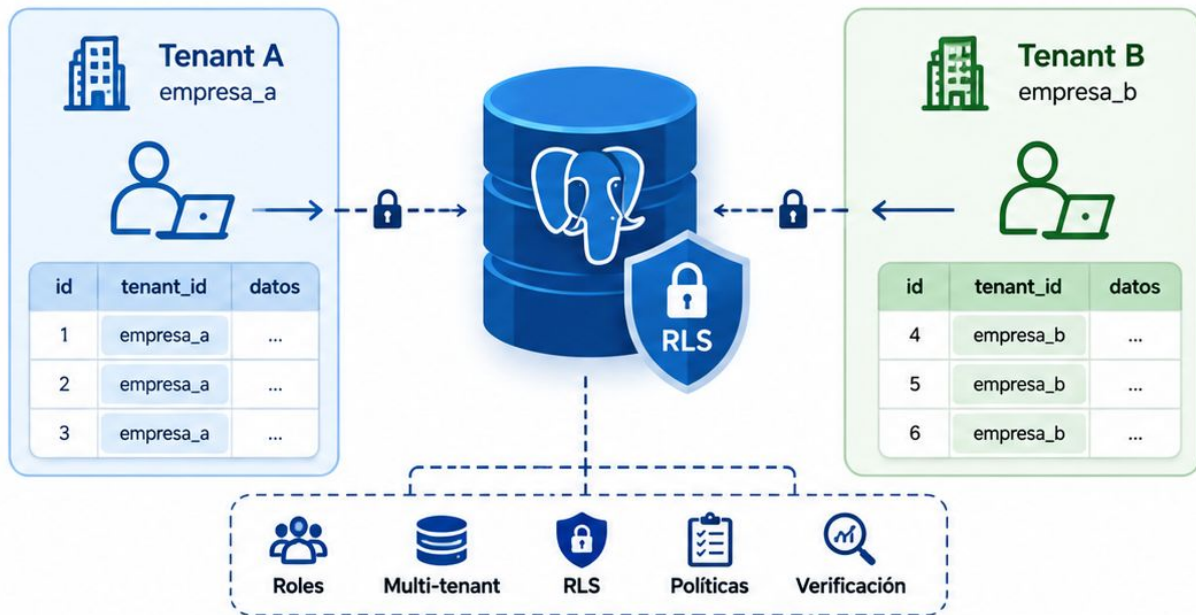


Muchas gracias ¿Dudas?

Práctica

Práctica Guiada con Row Level Security

Construir una base multi-tenant y verificar su aislamiento.



En esta práctica aplicamos lo visto para diseñar una base segura, no solo para que funcione, sino para que aisle datos entre tenants.

Práctica

Una aplicación compartida por varias organizaciones

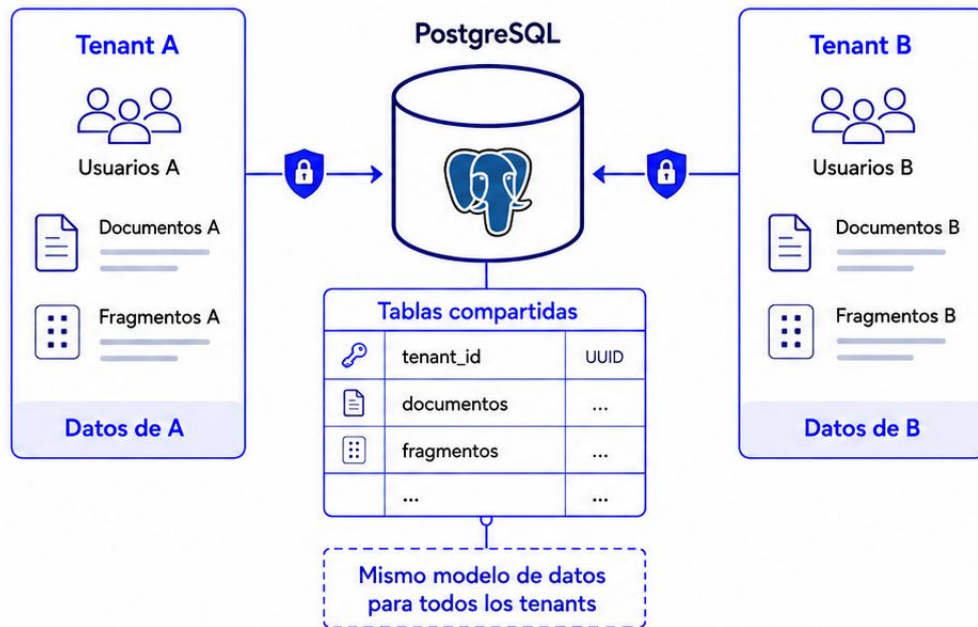
Vamos a trabajar con una aplicación que administra documentos para diferentes empresas.

Cada organización tendrá:

- usuarios;
- documentos;
- fragmentos;
- información propia.

Todas utilizarán las mismas tablas.

El objetivo será impedir que una organización consulte o modifique datos de otra.



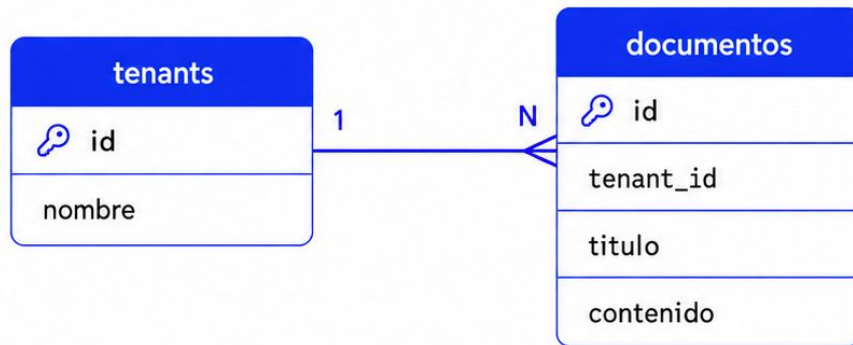
La práctica simula el mismo problema de aislamiento que aparece en una aplicación SaaS o RAG multi-tenant.

Práctica

Incorporar tenant al esquema

```
CREATE TABLE tenants (  
  id BIGSERIAL PRIMARY KEY,  
  nombre TEXT NOT NULL  
);  
  
CREATE TABLE documentos (  
  id BIGSERIAL PRIMARY KEY,  
  tenant_id BIGINT NOT NULL  
    REFERENCES tenants(id),  
  titulo TEXT NOT NULL,  
  contenido TEXT NOT NULL  
);
```

Tenant 1:N Documentos



Antes de aplicar seguridad debemos representar correctamente a quién pertenece cada dato.

Práctica

Crear datos de las organizaciones

Vamos a insertar:

- Tenant 1;
- Tenant 2;
- varios documentos para cada uno.

Después ejecutaremos:

```
SELECT *  
FROM documentos;
```

Inicialmente podremos observar todos los registros.

Esto nos permite establecer el comportamiento de referencia antes de activar RLS.



documentos

id	tenant_id	titulo	contenido_resumen
1	tenant_1	Política de seguridad	Directrices de acceso seguro...
2	tenant_2	Manual de onboarding	Guía para nuevos empleados...
3	tenant_1	Informe trimestral Q1	Resultados financieros del Q1...
4	tenant_2	Estrategia de producto	Visión y roadmap del producto...
5	tenant_1	Plan de capacitación	Programa de formación 2024...
6	tenant_2	Reporte de incidentes	Incidentes registrados en mayo...

Antes de RLS: todos los registros visibles



Primero comprobamos el problema.

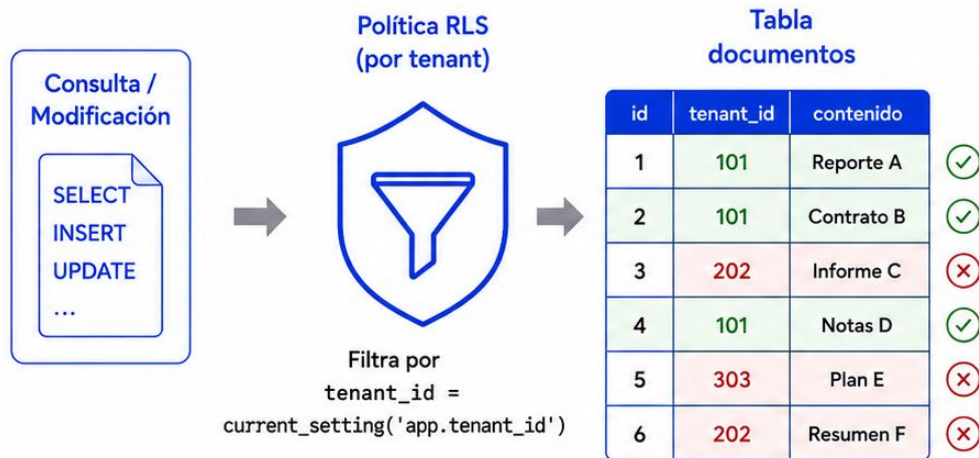
Después incorporamos el mecanismo de aislamiento.

Práctica

Crear una política por tenant

```
ALTER TABLE documentos  
ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY documentos_tenant  
ON documentos  
USING (  
    tenant_id =  
    current_setting('app.tenant_id')::BIGINT  
)  
WITH CHECK (  
    tenant_id =  
    current_setting('app.tenant_id')::BIGINT  
);
```



La misma regla controla qué datos podemos observar y qué datos podemos guardar.

Práctica

Ejecutar la misma consulta con contextos diferentes

```
BEGIN;
```

```
SET LOCAL app.tenant_id = '1';
```

```
SELECT *  
FROM documentos;
```

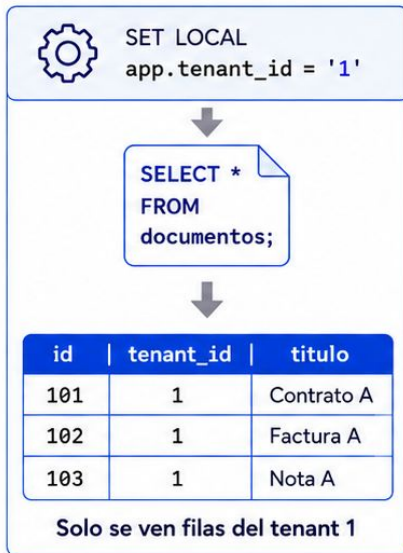
```
COMMIT;
```

Luego repetiremos con:

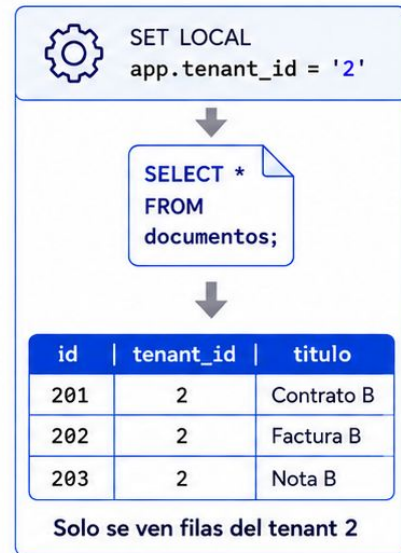
```
SET LOCAL app.tenant_id = '2';
```

Vamos a comparar los resultados.

Sesión / Contexto 1



Sesión / Contexto 2



La consulta es la misma.

Lo que cambia es el contexto autorizado.

Práctica

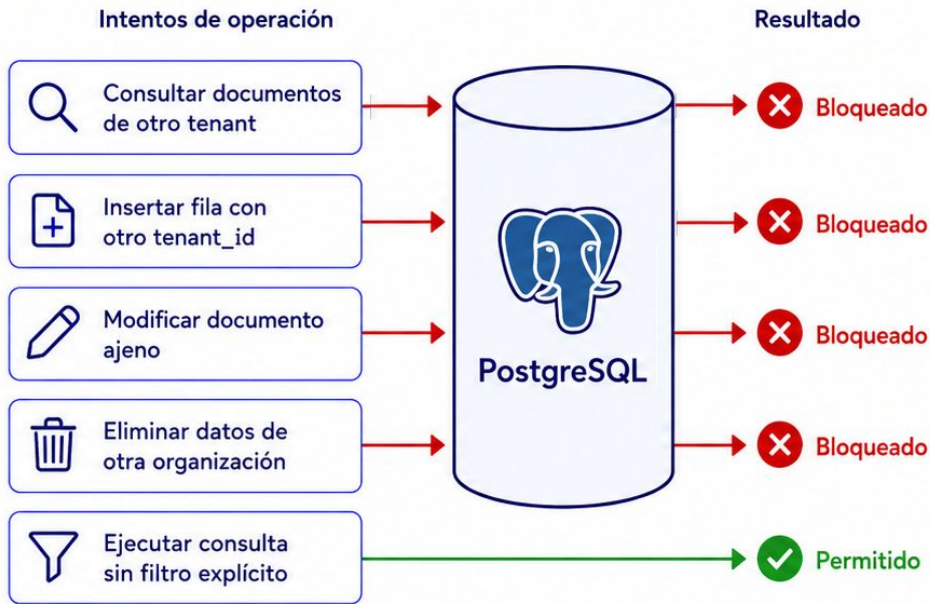
Probar operaciones que deberían fallar

Vamos a intentar:

- consultar documentos de otro tenant;
- insertar una fila con otro tenant_id;
- modificar un documento ajeno;
- eliminar información de otra organización;
- ejecutar consultas sin filtro explícito.

Finalmente vamos a discutir cómo este mismo mecanismo puede proteger:

- APIs;
- aplicaciones SaaS;
- búsquedas vectoriales;
- sistemas RAG;
- herramientas utilizadas por agentes.



La mejor forma de verificar una política de aislamiento es intentar acceder a datos que no deberían estar disponibles.